

UVM 기초

그림, 설명, 코드로 이해하는 검증환경

언어에서 검증환경까지

- 객체와 시간 → 생성과 연결 → 요청과 관찰 → 비교와 모델

SystemVerilog 기초 · UVM 환경 · Coverage · Callback · RAL

그림에서 동작을 읽고 코드로 확인한다

이 책의 구성

1~4장 · 언어와 시간

- 객체 참조, 데이터 구조, 무작위 조건, interface와 simulation timing

5~11장 · 검증환경 구성

- UVM 구조, 생성과 연결, 요청 전달, 설정과 component 통신

12~16장 · 검증과 확장

- 관찰과 비교, 로그, coverage와 assertion, Callback과 RAL

목차

장별 구성

- 01 객체지향 문법
- 02 데이터 구조와 프로세스 통신
- 03 Randomization과 Constraint
- 04 Interface와 Simulation Timing
- 05 UVM 개요와 전체 구조
- 06 uvm_object와 uvm_component
- 07 Factory와 Utility Macro
- 08 Phase와 Objection

목차

장별 구성

09 Sequence와 Driver Handshake

10 Config DB와 Virtual Interface

11 TLM과 Analysis

12 Monitor와 Scoreboard

13 Report와 디버깅

14 Coverage와 Assertion

15 Callback

16 Register Abstraction Layer

1장

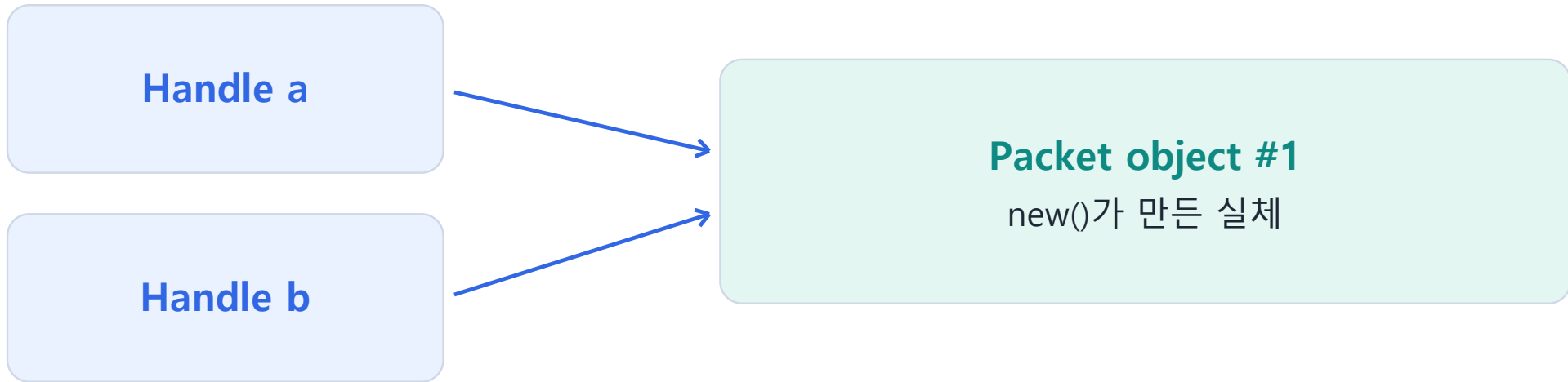
객체지향 문법

소개

- SystemVerilog 검증 코드는 class로 데이터와 동작을 묶는다.
- 설계도인 class, 생성된 object, 이를 참조하는 handle을 구분하면 상속과 UVM의 객체 전달을 이해할 수 있다.

구성 요소와 데이터 흐름

구조와 흐름



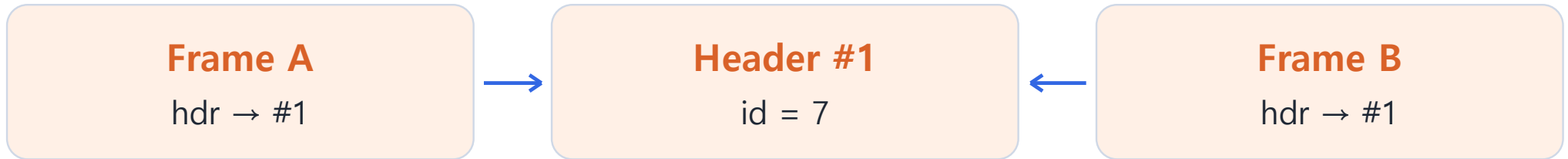
그림을 읽는 기준

- SystemVerilog 검증 코드는 class로 데이터와 동작을 묶는다.
- 설계도인 class, 생성된 object, 이를 참조하는 handle을 구분하면 상속과 UVM의 객체 전달을 이해할 수 있다.

참조와 복사에서 공유되는 객체를 구분한다

핵심 개념

SHALLOW COPY



DEEP COPY



Class / object / handle

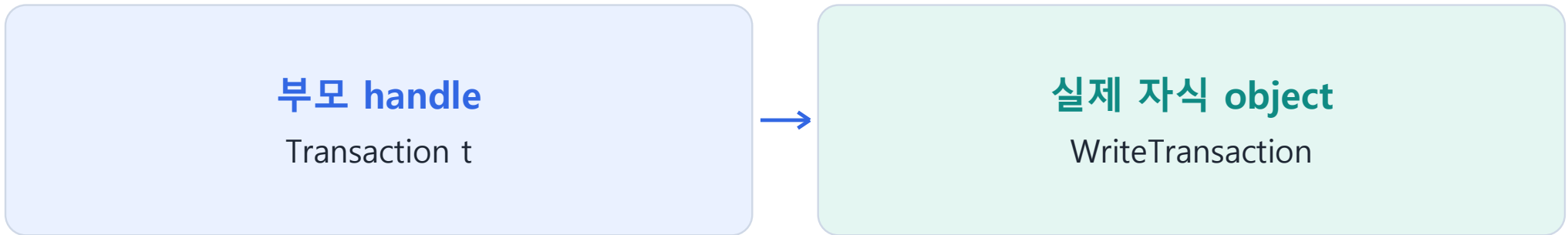
- class는 설계도, object는 실체, handle은 참조다.
- 선언만 한 handle은 null이며 new()가 객체를 만든다.

공유와 복사

- `b = a`는 handle 대입이다.
- 얇은 복사는 중첩 handle을 공유하고, 깊은 복사는 내부 객체까지 독립시킨다.

부모 handle로 자식의 동작을 선택한다

핵심 개념



virtual 호출은 실제 객체의 구현을 선택

상속과 다형성

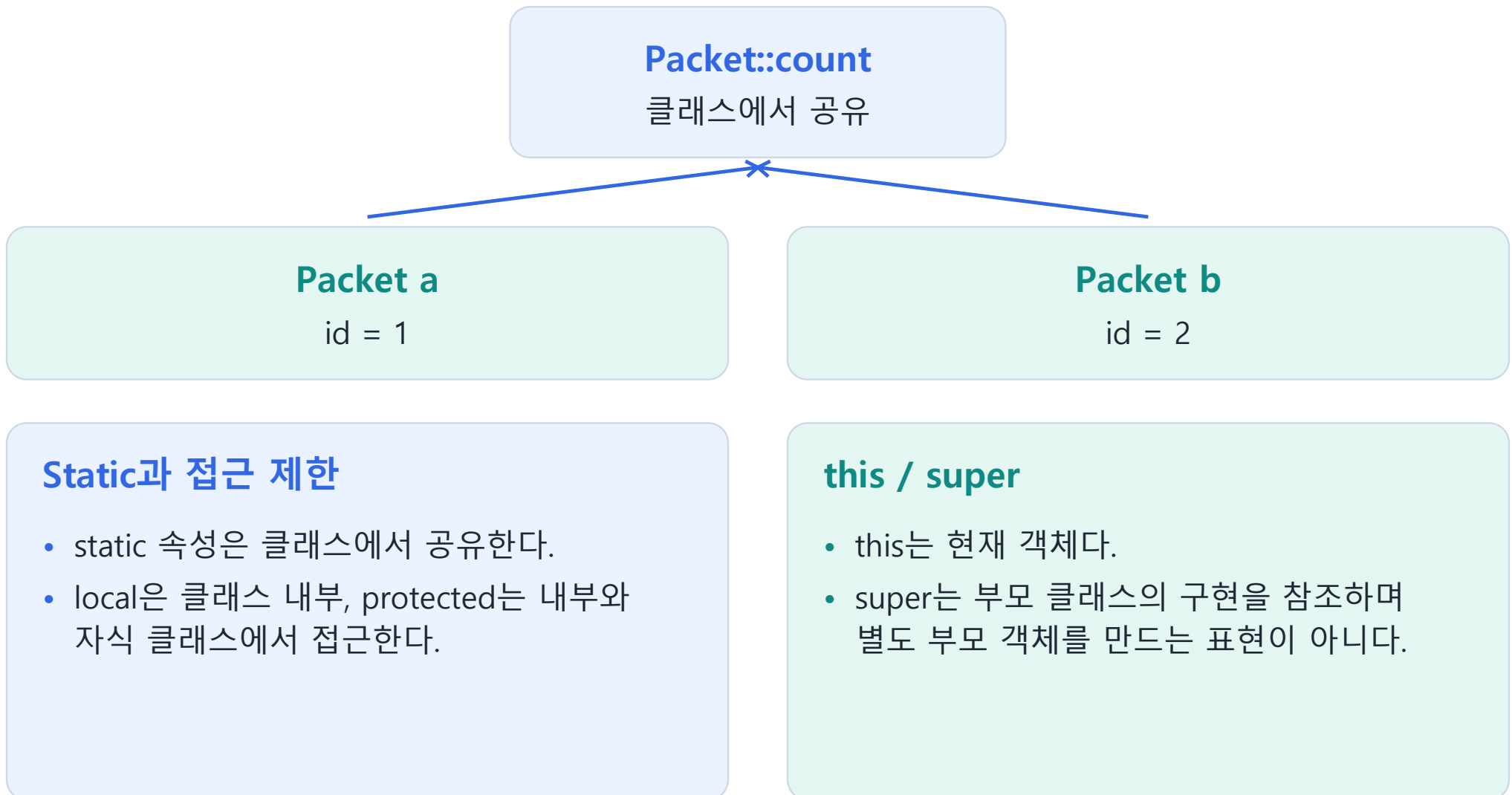
- extends는 클래스 상속이다.
- virtual 메서드는 handle의 선언 타입보다 실제 객체 타입에 따라 구현을 선택한다.

추상·매개변수 클래스

- virtual class는 직접 생성할 수 없다.
- type parameter를 사용하면 같은 구조를 서로 다른 자료형에 적용한다.

객체의 값과 클래스의 공유 값을 구분한다

핵심 개념



코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

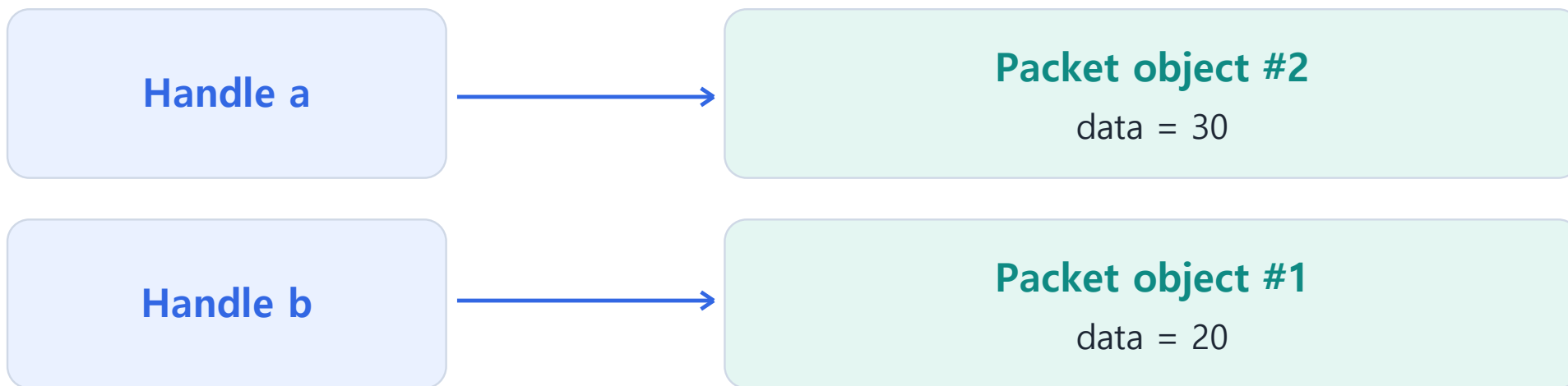
```
01 Packet a, b;  
02 a = new(10);  
03 b = a;  
04 b.data = 20;  
05 // a.data도 20: 같은 객체를 참조  
06 a = new(30);  
07 // b는 기존 객체를 계속 참조
```

코드를 읽는 기준

- 첫 new() 뒤에는 객체 하나를 공유한다.
- 두 번째 new()는 새 객체를 만들고 a만 그 객체를 참조하게 한다.

새 객체 생성은 a의 연결만 바꾼다

코드의 동작



결과 해설

- 첫 new() 뒤에는 객체 하나를 공유한다.
- 두 번째 new()는 새 객체를 만들고 a만 그 객체를 참조하게 한다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Handle 대입은 객체 복사가 아니다.

주의 2

- 상속 관계와 생성된 객체 사이의 포함 관계를 구분한다.

주의 3

- 부모 handle로 자식 전용 멤버를 사용하려면 성공한 \$cast 결과가 필요하다.

이 장의 핵심

- Class / object / handle
- 공유와 복사
- 상속과 다형성
- 추상·매개변수 클래스
- Static과 접근 제한
- this / super

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- `a = new(); b = a;` 이후 객체는 몇 개인가?

문제 2

- 폭 8과 폭 16의 매개변수 클래스는 같은 타입인가?

문제 3

- 부모 handle의 virtual 호출은 무엇을 기준으로 선택하는가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 하나다.
- 두 handle이 같은 객체를 참조한다.

해설 2

- 서로 다른 specialization 타입이다.

해설 3

- 실제 객체 타입의 구현을 선택한다.

2장

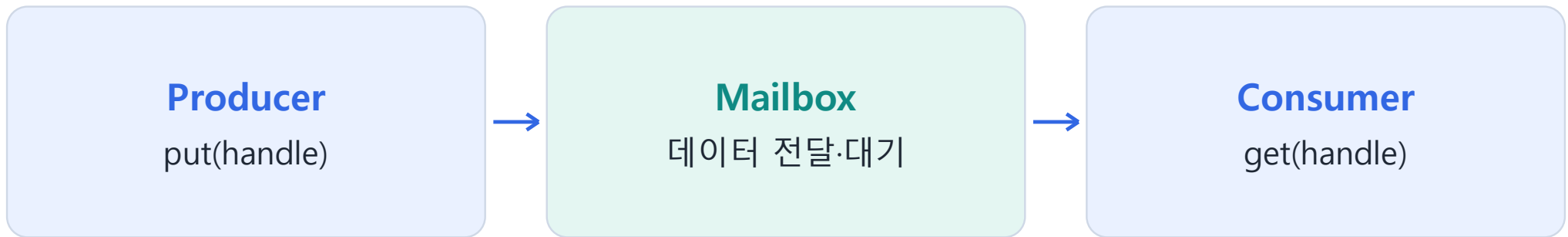
데이터 구조와 프로세스 통신

소개

- 검증환경에는 데이터 저장과 동시 실행을 위한 구조가 필요하다.
- 배열과 queue는 값을 보관하고, event-semaphore-mailbox는 프로세스 사이의 통지를 전달하거나 접근을 조정한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- 검증환경에는 데이터 저장과 동시 실행을 위한 구조가 필요하다.
- 배열과 queue는 값을 보관하고, event-semaphore-mailbox는 프로세스 사이의 통지를 전달하거나 접근을 조정한다.

배열의 비트 폭과 원소 수를 읽는다

핵심 개념

logic [7:0] memory [0:3];

memory[0]

[7:0] = 8 bits

memory[1]

[7:0] = 8 bits

memory[2]

[7:0] = 8 bits

memory[3]

[7:0] = 8 bits

이름 앞: 비트 폭 / 이름 뒤: 원소 개수

Packed / unpacked

- 이름 앞 차원은 packed 비트 묶음, 이름 뒤 차원은 unpacked 원소 배열이다.
- logic [7:0] a[4]는 8비트 원소 4개다.

배열과 queue

- 고정 배열은 크기가 정해져 있다.
- 동적 배열은 new[size], queue는 push/pop으로 크기를 바꾼다.

ID로 찾는 데이터와 병렬 실행을 구분한다

핵심 개념

ID로 찾기

expected[42]

Transaction #42

연관 배열과 자료형

- Associative array는 key로 저장값을 찾는다.
- enum·typedef·struct는 의미 있는 타입과 데이터 묶음을 만든다.

병렬 프로세스

———— 5 ns

———— 10 ns

———— 15 ns

프로세스 실행

- 일반 task 호출은 순차 실행이다.
- fork/join은 병렬 실행 후 모두 대기, join_any는 하나, join_none은 대기 없이 진행한다.

통지, 접근 제어, 데이터 전달을 구분한다

핵심 개념



Semaphore는 접근권, event는 발생 통지

Event와 semaphore

- Event는 발생 통지이며 데이터 저장소가 아니다.
- Semaphore는 key 획득·반환으로 공유 자원의 접근을 제어한다.

Mailbox와 handle

- Mailbox는 put/get으로 데이터를 전달한다.
- Class 객체를 넣으면 handle이 전달되므로 독립 snapshot이 필요하면 복사 정책을 둔다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 mailbox #(Packet) mbx = new();
02 Packet sent, received;
03 sent = new(7);
04 mbx.put(sent);
05 mbx.get(received);
06 sent.data = 9;
07 // received.data도 9
```

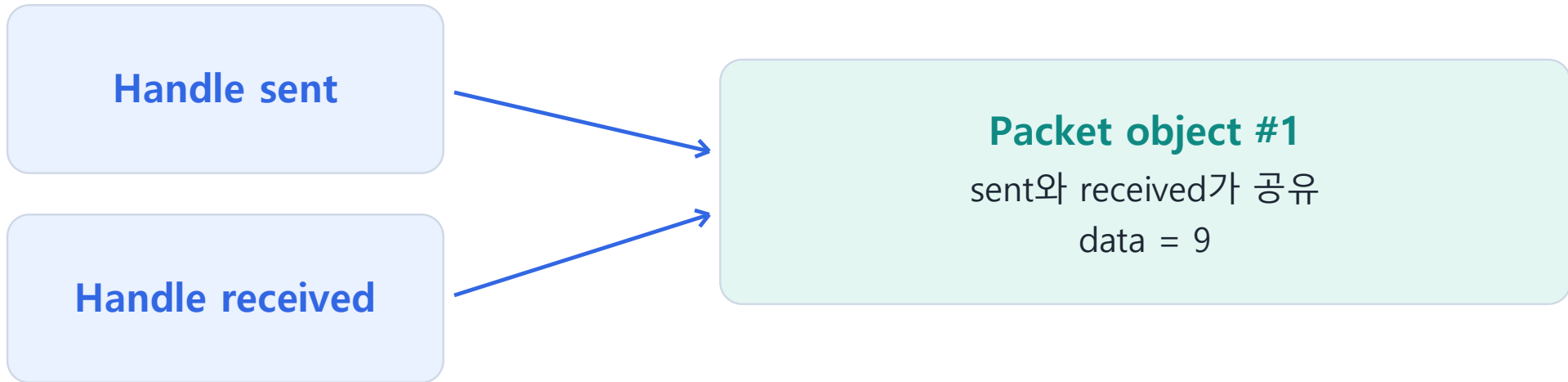
코드를 읽는 기준

- Mailbox가 전달하는 것은 Packet의 handle이다.
- 수신 후에도 두 handle은 같은 객체를 가리킨다.

Mailbox로 전달해도 같은 객체를 참조한다

코드의 동작

Mailbox를 지나도 같은 객체의 handle



결과 해설

- Mailbox가 전달하는 것은 Packet의 handle이다.
- 수신 후에도 두 handle은 같은 객체를 가리킨다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Event를 과거 발생 이력을 저장하는 queue처럼 사용하지 않는다.

주의 2

- disable fork는 현재 프로세스의 활성 자손에 영향을 준다.

주의 3

- 시간 대기를 허용하는 task라는 이유만으로 병렬 실행되지 않는다.

이 장의 핵심

- Packed / unpacked
- 배열과 queue
- 연관 배열과 자료형
- 프로세스 실행
- Event와 semaphore
- Mailbox와 handle

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- logic [7:0] a[4]의 원소 수와 원소 폭은?

문제 2

- 10ns와 20ns task를 fork/join으로 실행하면?

문제 3

- Mailbox 수신 객체를 독립시키려면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 4개, 각 8비트다.

해설 2

- 같이 시작한다면 20ns 뒤 모두 완료된다.

해설 3

- 새 객체에 필요한 필드와 중첩 객체를 복사한다.

3장

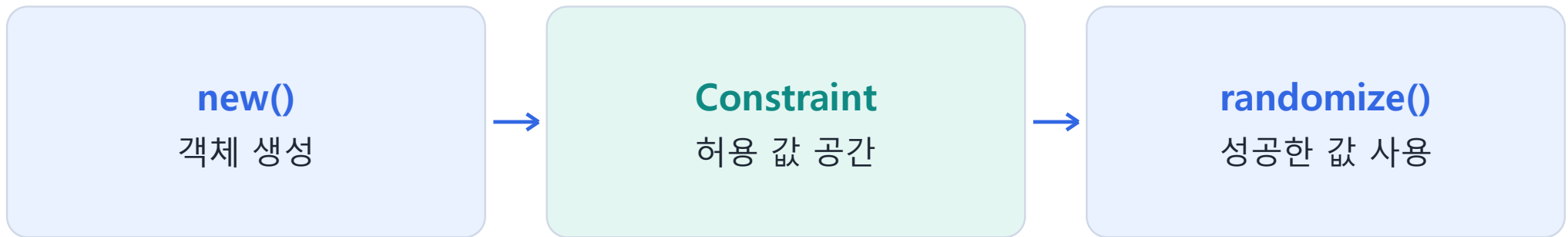
Randomization과 Constraint

소개

- Constrained random은 허용된 값 공간에서 자극을 선택한다.
- 무작위 값 자체보다 유효한 조건, 경계 상황, 실패 처리와 재현 가능한 seed 관리가 중요하다.

구성 요소와 데이터 흐름

구조와 흐름



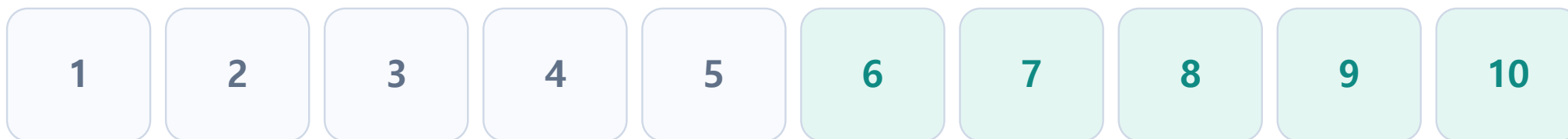
그림을 읽는 기준

- Constrained random은 허용된 값 공간에서 자극을 선택한다.
- 무작위 값 자체보다 유효한 조건, 경계 상황, 실패 처리와 재현 가능한 seed 관리가 중요하다.

기존 조건과 추가 조건을 함께 만족한다

핵심 개념

기존 조건 [1:10] $\cap$ 추가 조건 `data > 5`



성공한 `data`는 {6, 7, 8, 9, 10} 중 하나

rand / randc

- `rand`는 무작위 값, `randc`는 변수의 유효 값 공간을 순환한다.
- 매번 새 객체를 만들면 이전 객체의 순환 상태는 이어지지 않는다.

조건과 충돌

- `Constraint`는 대입 순서가 아닌 관계식이다.
- `Inline constraint`는 기존 조건에 더해지며 충돌하면 `randomize()`가 실패한다.

범위의 가중치와 기본 조건을 해석한다

핵심 개념

$:=$ 각 값에 적용

$[1:3] := 6$

$:/$ 범위 전체에 적용

$[1:3] :/ 6$

가중치: 1·2·3 각각 6 / 각각 2

범위와 확률

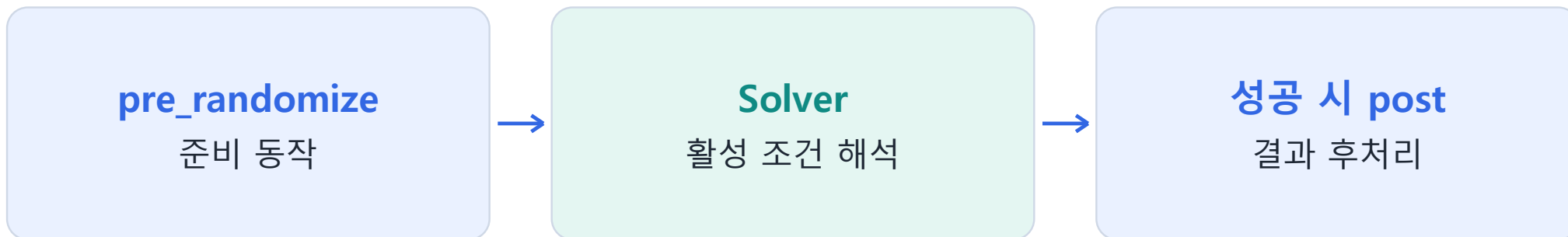
- inside는 허용 집합, dist는 가중치를 정한다.
- $:=$ 는 각 값에, $:/$ 는 범위 전체에 가중치를 배분한다.

선택 순서와 기본값

- solve before는 선택 확률에 영향을 준다.
- soft constraint는 더 강한 조건과 충돌하면 양보하는 기본 조건이다.

무작위화의 활성화 조건과 hook을 확인한다

핵심 개념



동작 제어와 hook

- rand_mode는 변수의 무작위화, constraint_mode는 조건 활성화를 제어한다.
- pre/post_randomize는 호출 전후 동작을 추가한다.

상속과 실패 처리

- 같은 이름의 constraint는 자식에서 재정의할 수 있다.
- 실패를 검사하고 성공한 값만 자극으로 사용한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 class item;  
02     rand bit [7:0] data;  
03     constraint valid_c { data inside {[1:10]}; }  
04 endclass  
05 item tr = new();  
06 if (!tr.randomize() with { data > 5; })  
07     $fatal(1, "Randomization failed");
```

코드를 읽는 기준

- 기존 범위 1~10과 inline 조건 `data > 5`를 함께 만족해야 하므로 최종 허용 범위는 6~10이다.

Randomize 성공 시 6~10 중 값을 선택한다

코드의 동작

기본 조건 [1:10] $\cap$ 추가 조건 `data > 5`



성공한 `data`는 {6, 7, 8, 9, 10} 중 하나

결과 해설

- 기본 범위 1~10과 inline 조건 `data > 5`를 함께 만족해야 하므로 최종 허용 범위는 6~10이다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- 생성만으로 rand 필드가 자동 randomize되지는 않는다.

주의 2

- post_randomize에서 값을 바꾸면 조건을 어긴 값이 될 수 있다.

주의 3

- 실패 후 기존 값을 정상 자극인 것처럼 사용하지 않는다.

이 장의 핵심

- rand / randc
- 조건과 충돌
- 범위와 확률
- 선택 순서와 기본값
- 동작 제어와 hook
- 상속과 실패 처리

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- data inside {[1:10]}에 data > 20을 추가하면?

문제 2

- rand_mode(0)는 constraint도 끄는가?

문제 3

- Seed를 남기는 이유는?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 조건이 충돌하므로 실패한다.

해설 2

- 아니다.
- 조건은 남아 고정된 변수값을 제한할 수 있다.

해설 3

- 실패한 무작위 시나리오를 재현하기 위해서다.

4장

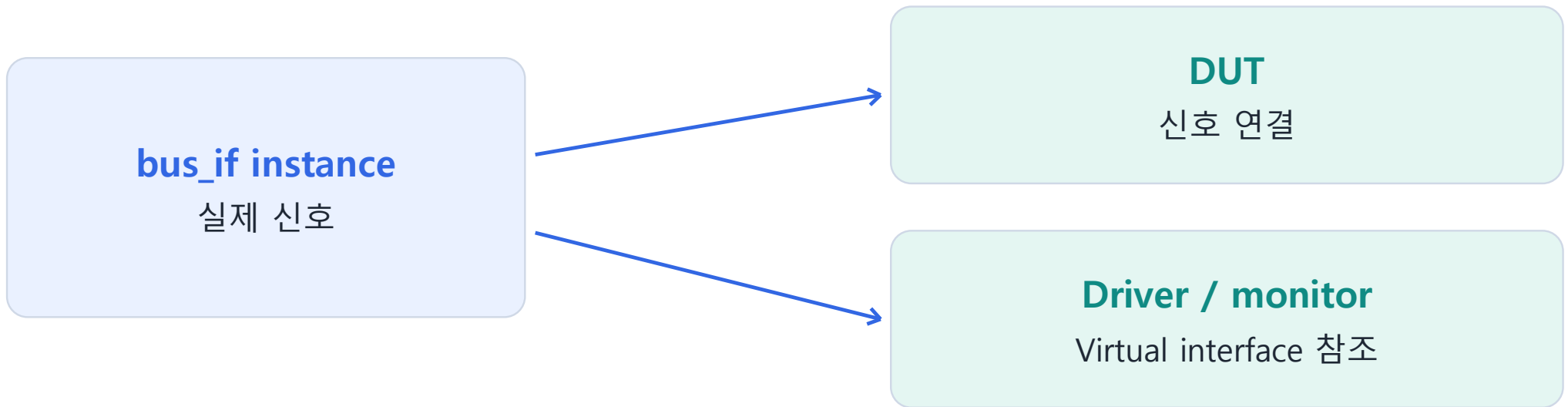
Interface와 Simulation Timing

소개

- Class 기반 검증 코드가 DUT의 신호에 접근하려면 interface를 참조해야 한다.
- 올바른 값뿐 아니라 구동과 관찰 시점을 맞추는 것이 race를 피하고 정확한 검증을 만드는 핵심이다.

구성 요소와 데이터 흐름

구조와 흐름

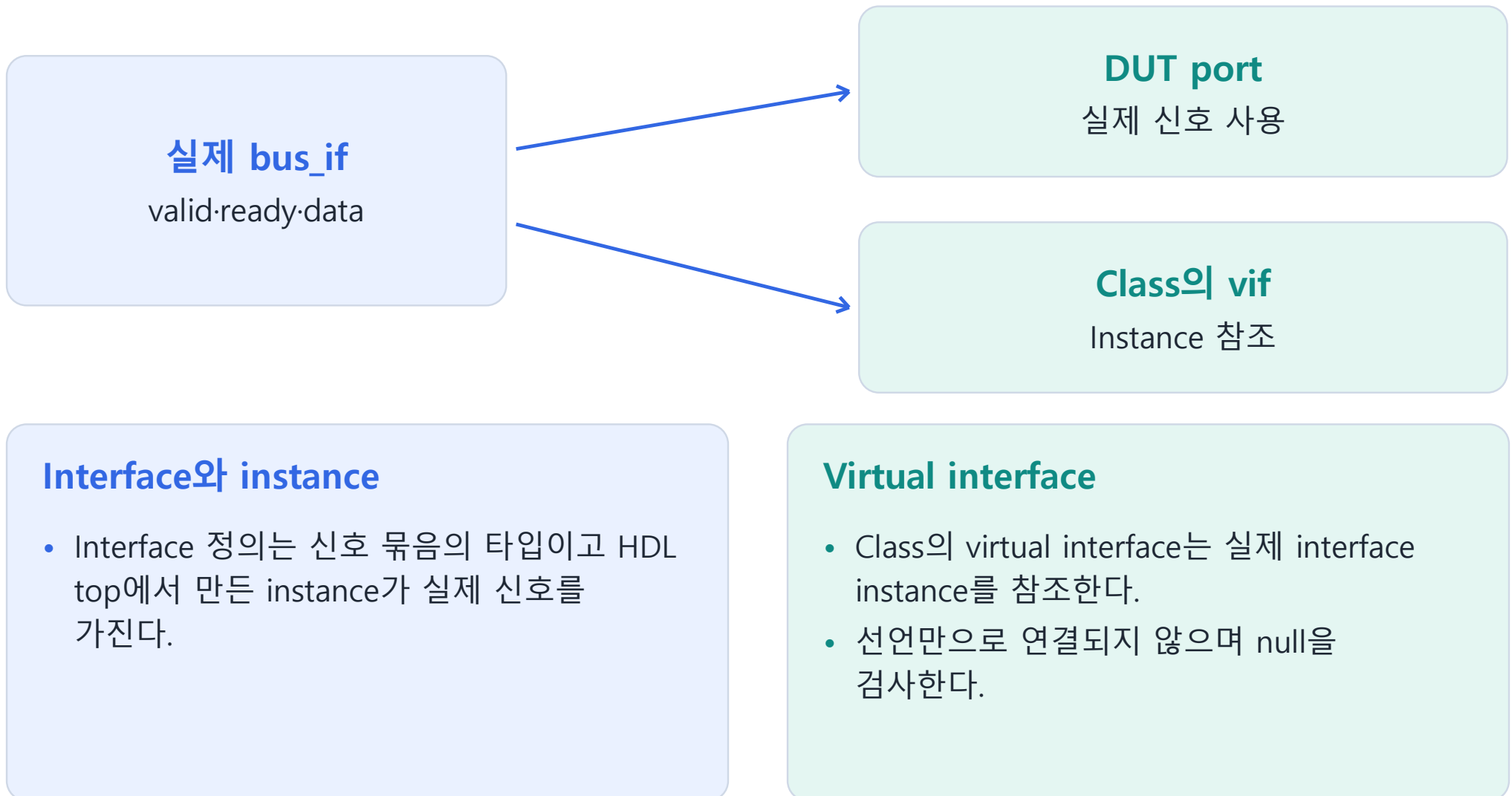


그림을 읽는 기준

- Class 기반 검증 코드가 DUT의 신호에 접근하려면 interface를 참조해야 한다.
- 올바른 값뿐 아니라 구동과 관찰 시점을 맞추는 것이 race를 피하고 정확한 검증을 만드는 핵심이다.

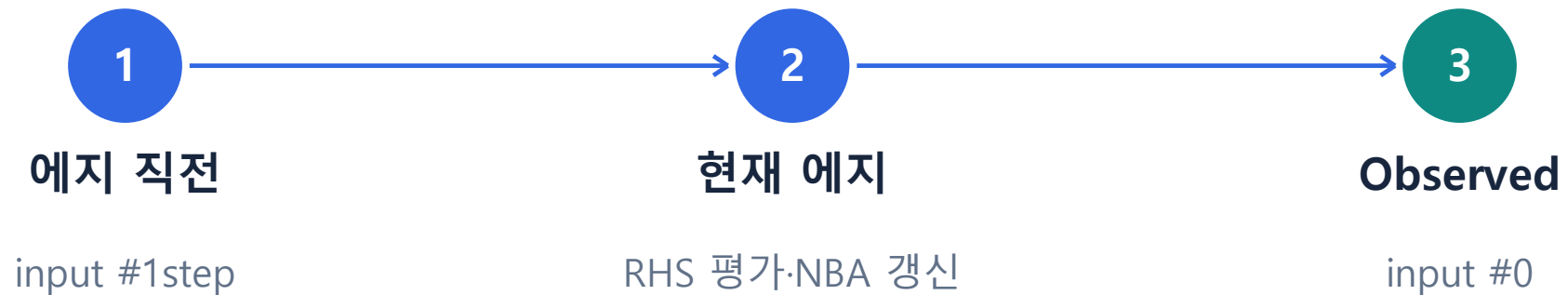
실제 interface와 virtual 참조를 연결한다

핵심 개념



같은 에지 안에서도 관찰 시점이 다르다

핵심 개념



Modport / clocking

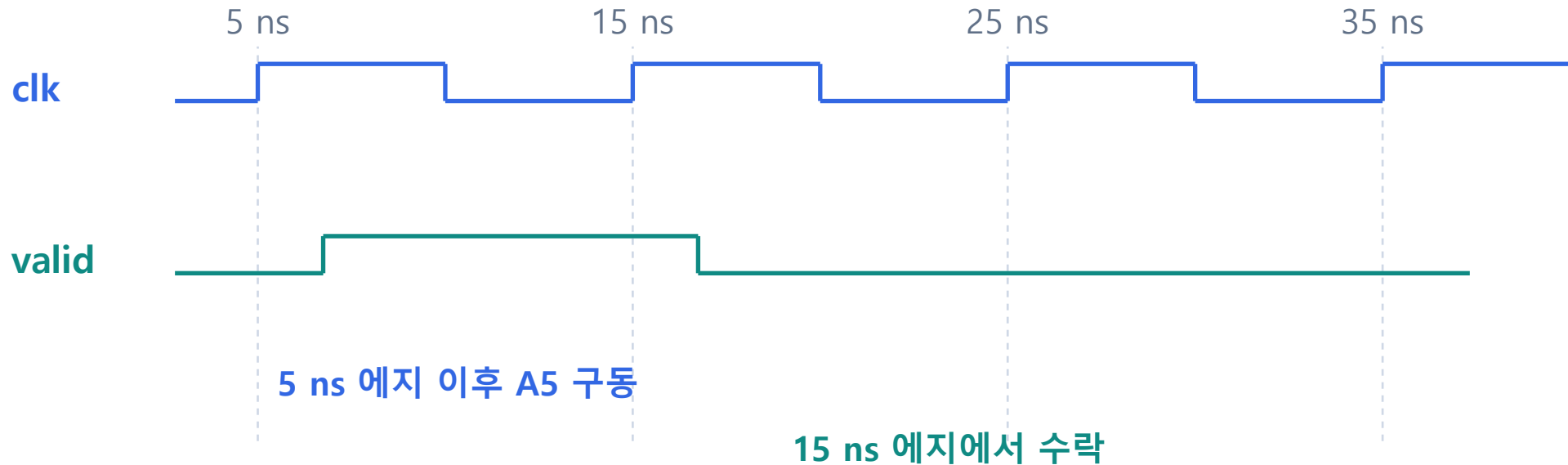
- Modport는 접근 방향·뷰를 정한다.
- Clocking block은 이벤트와 input/output skew로 구동·관찰 시점을 표현한다.

Blocking / NBA

- Blocking은 해당 프로세스에서 즉시 대입한다.
- Nonblocking은 RHS를 평가하고 NBA 영역에서 갱신하므로 같은 에지의 관찰 시점이 중요하다.

구동 에지와 수락 에지를 구분한다

핵심 개념 · 그림 가정: posedge 수락, ready=1, clocking output #0



Sampling 기준

- Clocking input #1step은 에지 직전 안정값, input #0은 같은 에지의 Observed 영역 값에 대응한다.
- 프로토콜 계약에 맞춰 선택한다.

요청과 응답 timing

- 요청 수락 때의 상태와 등록된 출력의 유효 시점은 다를 수 있다.
- Arbitrary #delay보다 사양과 scheduling을 기준으로 관찰한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 interface bus_if(input logic clk);  
02     logic valid, ready;  
03     logic [7:0] data;  
04     clocking drv_cb @(posedge clk);  
05         default input #1step output #0;  
06         input ready;  
07         output valid, data;  
08     endclocking  
09 endinterface
```

코드를 읽는 기준

- Driver는 ready의 sampling과 valid/data 구동을 clocking 규칙으로 표현한다.
- DUT가 값을 수락하는 예지는 별도의 프로토콜 계약에 따른다.

Clocking block이 구동과 관찰의 기준을 정한다

코드의 동작

input ready

에지 직전의 안정값 관찰

output valid, data

현재 clocking 이벤트에 구동

실제 수락 시점은 DUT의 sampling 계약으로 판단

결과 해설

- Driver는 ready의 sampling과 valid/data 구동을 clocking 규칙으로 표현한다.
- DUT가 값을 수락하는 에지는 별도의 프로토콜 계약에 따른다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Clocking input #0과 절차문의 #0 delay는 같은 의미가 아니다.

주의 2

- 같은 posedge에서 구동·관찰하면 자동으로 안전한 것은 아니다.

주의 3

- Interface parameter와 modport도 virtual interface 타입에 맞춘다.

이 장의 핵심

- Interface와 instance
- Virtual interface
- Modport / clocking
- Blocking / NBA
- Sampling 기준
- 요청과 응답 timing

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Virtual interface 선언만 하면 실제 신호에 연결되는가?

문제 2

- Modport와 clocking block의 역할 차이는?

문제 3

- 등록 출력은 요청 수락 에지의 이전 rdata와 비교해도 되는가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- 실제 instance 참조를 전달해야 한다.

해설 2

- 접근 뷰·방향과 구동·sampling timing이다.

해설 3

- 출력 latency와 유효 시점을 먼저 확인해야 한다.

5장

UVM 개요와 전체 구조

소개

- UVM은 검증환경의 역할과 실행·생성·통신 방식을 표준화한다.
- 요청을 만드는 흐름과 실제 동작을 관찰하고 검사하는 흐름을 구분하면 재사용 가능한 구조를 이해하기 쉽다.

구성 요소와 데이터 흐름

구조와 흐름

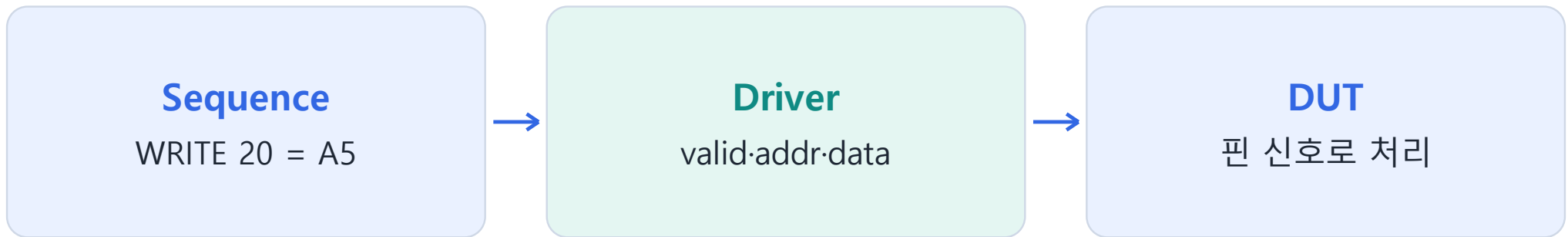


그림을 읽는 기준

- UVM은 검증환경의 역할과 실행·생성·통신 방식을 표준화한다.
- 요청을 만드는 흐름과 실제 동작을 관찰하고 검사하는 흐름을 구분하면 재사용 가능한 구조를 이해하기 쉽다.

검증 의도를 실제 신호 동작으로 바꾼다

핵심 개념



검증의 목적

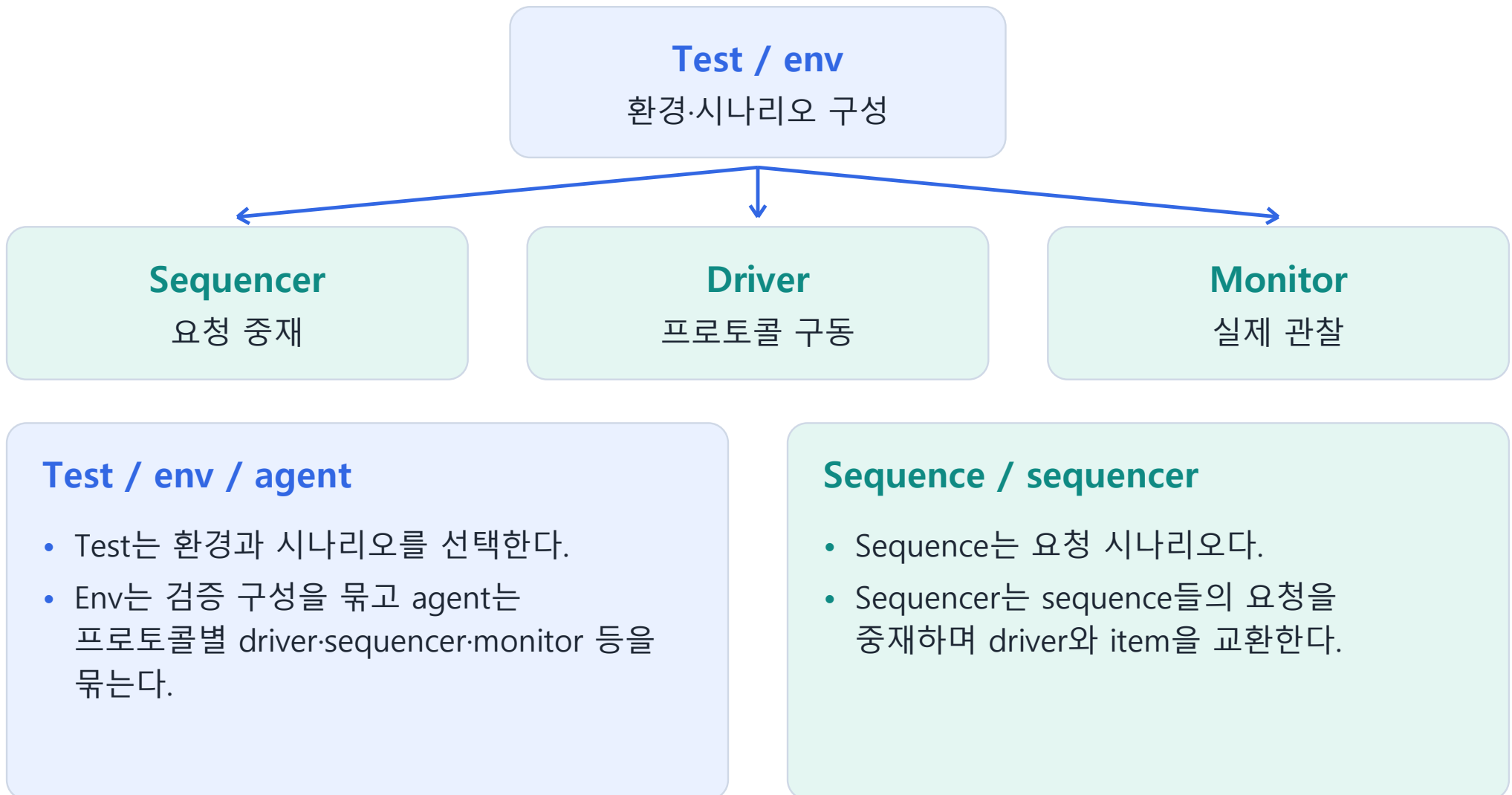
- Directed test는 특정 상황을 명확히 만든다.
- Constrained random은 유효 공간을 넓게 탐색하며 checker와 coverage가 필요하다.

Transaction 수준

- Transaction은 의미 있는 동작 한 건을 표현한다.
- Driver가 이를 pin 동작으로 바꾸고 monitor가 pin 관찰을 transaction으로 재구성한다.

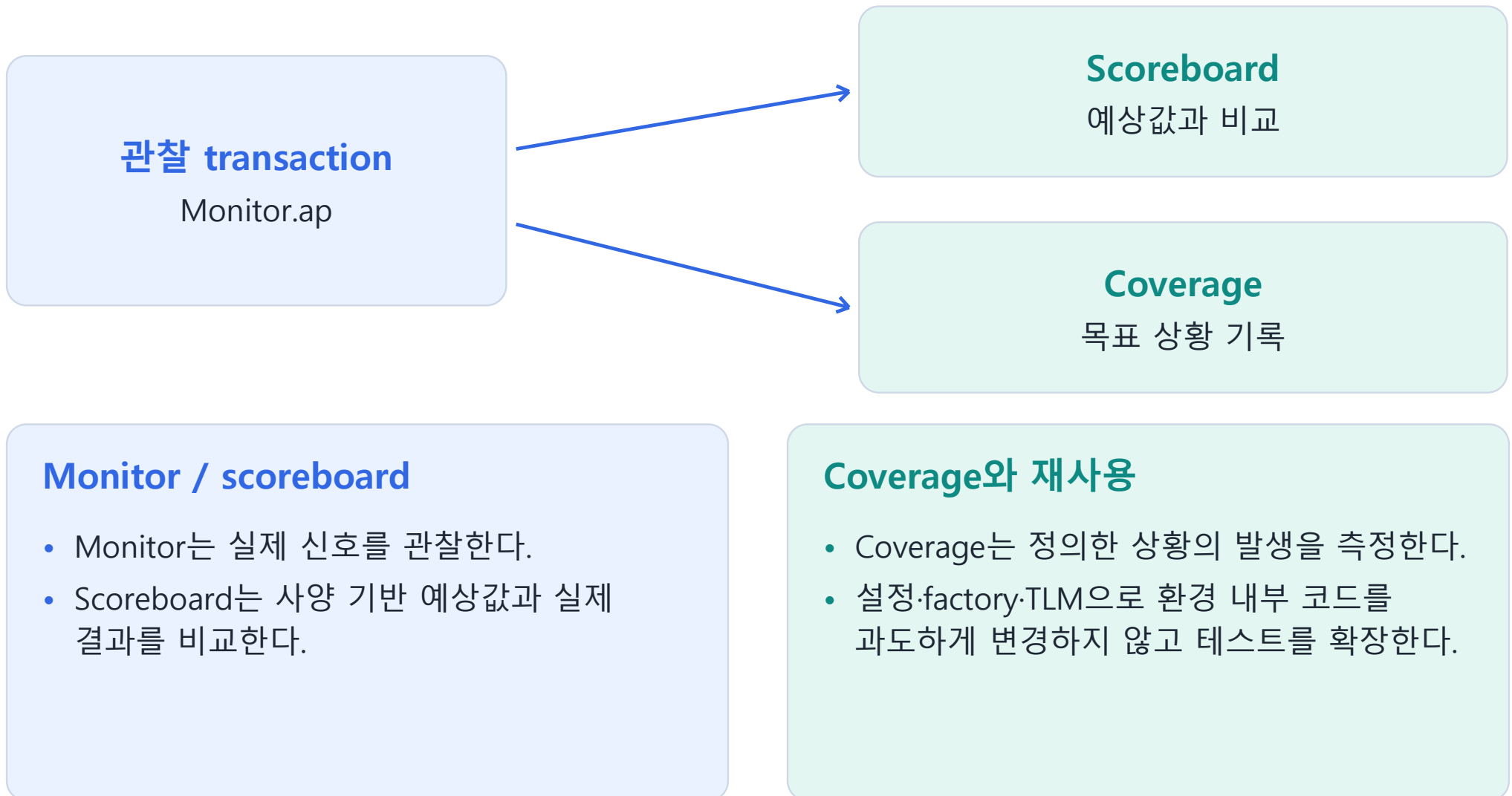
환경 구성과 요청 중재의 역할을 나눈다

핵심 개념



관찰 결과를 비교와 coverage에 전달한다

핵심 개념



코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 // agent의 connect_phase
02 drv.seq_item_port.connect(sqr.seq_item_export);
03 // env의 connect_phase
04 agent.mon.ap.connect(sb.analysis_imp);
05 agent.mon.ap.connect(cov.analysis_export);
```

코드를 읽는 기준

- 요청 전달 경로와 관찰 결과의 방송 경로를 각각 연결한다.
- 연결만으로 sequence가 시작되거나 transaction이 발행되지는 않는다.

연결된 요청 경로와 관찰 경로를 따라간다

코드의 동작



결과 해설

- 요청 전달 경로와 관찰 결과의 방송 경로를 각각 연결한다.
- 연결만으로 sequence가 시작되거나 transaction이 발행되지는 않는다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Driver의 의도만으로 DUT 실제 동작을 확인했다고 판단하지 않는다.

주의 2

- Coverage가 채워져도 올바른 결과가 보장되지는 않는다.

주의 3

- Sequencer와 sequence를 같은 대상으로 생각하지 않는다.

이 장의 핵심

- 검증의 목적
- Transaction 수준
- Test / env / agent
- Sequence / sequencer
- Monitor / scoreboard
- Coverage와 재사용

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Driver와 monitor의 차이는?

문제 2

- Passive agent에는 무엇이 필요한가?

문제 3

- Scoreboard의 예상값은 어디서 오는가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 요청에 따라 신호를 구동하는 역할과 실제 신호를 관찰하는 역할이다.

해설 2

- 기본적으로 관찰을 위한 monitor가 필요하다.

해설 3

- 사양 기반 reference model과 관찰한 입력에서 만든다.

6장

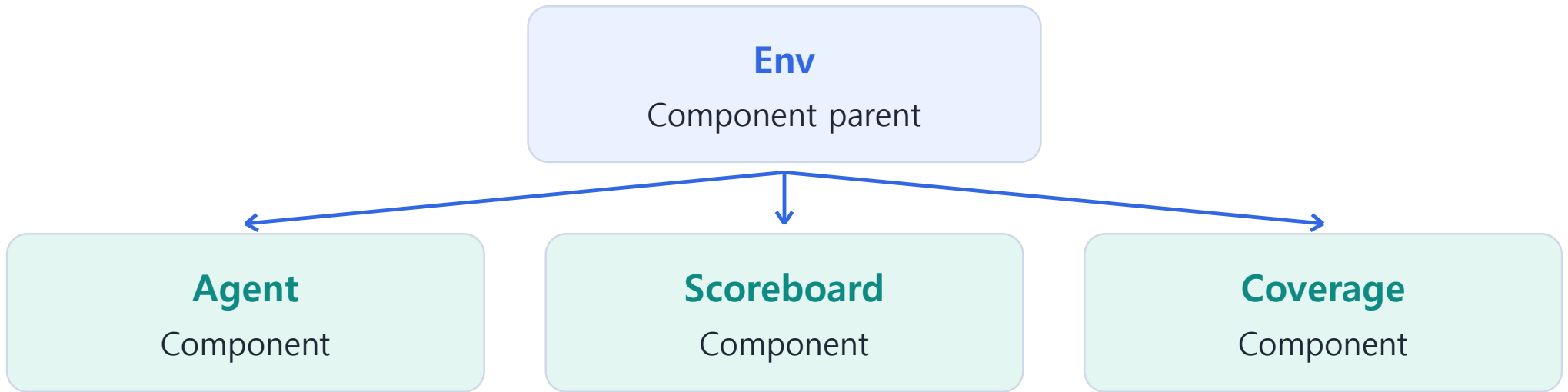
uvm_object와 uvm_component

소개

- UVM의 데이터를 표현하는 객체와 검증 계층을 이루는 component는 생성 방식과 실행 책임이 다르다.
- Class 상속, component parent, factory context를 서로 구분해야 한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- UVM의 데이터를 표현하는 객체와 검증 계층을 이루는 component는 생성 방식과 실행 책임이 다르다.
- Class 상속, component parent, factory context를 서로 구분해야 한다.

Object 생성과 sequence 실행을 구분한다

핵심 개념

Object 계열

Item·sequence

Component 계열

Driver·monitor·env

Sequence: create() → start() → body()

실무적인 계열 구분

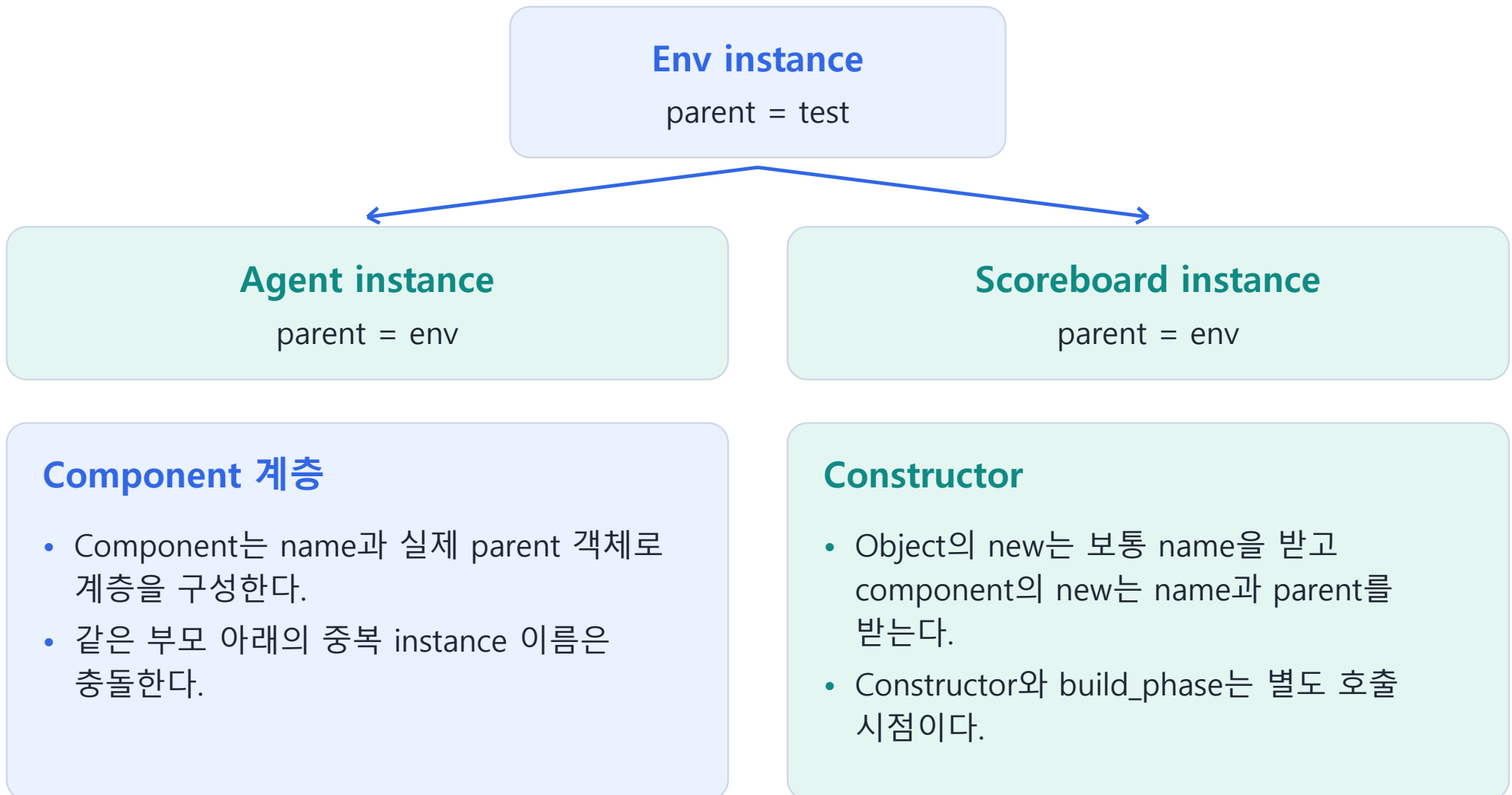
- Item·sequence는 object 계열, driver·monitor·env는 component 계열이다.
- Component도 상속상 uvm_object의 후손이다.

Object 실행

- Object는 component hierarchy에 등록되지 않는다.
- Sequence 생성만으로 body()가 실행되지 않으며 start()가 필요하다.

Component의 parent가 계층을 만든다

핵심 개념



타입 상속과 instance 배치는 다르다

핵심 개념

타입 상속

fifo_driver extends uvm_driver

Instance 배치

agent → drv

상속 관계와 parent 관계는 별도로 결정

상속과 배치

- extends는 타입 사이의 상속이다.
- create("drv", this)는 생성된 component의 parent 관계를 정한다.

이름과 context

- get_name은 leaf 이름, get_full_name은 전체 경로다.
- Object create의 parent 인수는 factory context이며 component 자식 등록이 아니다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 // Component의 생성
02 drv = fifo_driver::type_id::create("drv", this);
03 // Object의 생성: this는 factory context
04 item = fifo_item::type_id::create("item", this);
05 // Sequence 실행
06 seq = write_sequence::type_id::create("seq");
07 seq.start(sqr);
```

코드를 읽는 기준

- 같은 create 호출 모양이어도 registry 계열에 따라 parent의 의미가 다르다.
- Sequence의 실행에는 start가 추가로 필요하다.

Create는 생성, start는 실행이다

코드의 동작



결과 해설

- 같은 create 호출 모양이어도 registry 계열에 따라 parent의 의미가 다르다.
- Sequence의 실행에는 start가 추가로 필요하다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Component parent는 클래스 이름이 아니라 생성된 부모 객체다.

주의 2

- Sequence에는 component phase가 자동 호출되지 않는다.

주의 3

- UVM root와 HDL top module을 구분한다.

이 장의 핵심

- 실무적인 계열 구분
- Object 실행
- Component 계층
- Constructor
- 상속과 배치
- 이름과 context

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Driver가 agent 아래에 있으면 agent를 상속한 것인가?

문제 2

- Object create에 this를 넘기면 component 자식이 되는가?

문제 3

- Sequence body는 언제 실행되는가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- 포함 계층과 클래스 상속은 다르다.

해설 2

- 아니다.
- Factory 검색 context로 사용된다.

해설 3

- start() 등 sequence 실행 절차가 시작될 때다.

7장

Factory와 Utility Macro

소개

- Factory는 생성 요청에 실제로 사용할 타입을 선택한다.
- Utility macro는 등록과 타입 기능을 제공하고, field 자동화와 do_* hook은 복사·비교·출력 정책을 구현한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- Factory는 생성 요청에 실제로 사용할 타입을 선택한다.
- Utility macro는 등록과 타입 기능을 제공하고, field 자동화와 do_* hook은 복사·비교·출력 정책을 구현한다.

Override는 이후의 생성 요청에 적용된다

핵심 개념



c = new()는 factory 규칙을 거치지 않음

Factory와 new

- Create는 factory 검색을 거쳐 선택된 constructor를 호출한다.
- 직접 new는 override 검색을 거치지 않는다.

Type / instance override

- Type override는 요청 타입을 대체한다.
- Instance override는 생성 context의 경로에 적용되며 일치하면 type 규칙보다 우선한다.

등록과 필드 처리 정책을 구분한다

핵심 개념

Utils macro

Factory 등록

Field macro

Copy·compare 등 필드 정책

clone = 새 바깥 객체 생성 + copy

Utils와 field

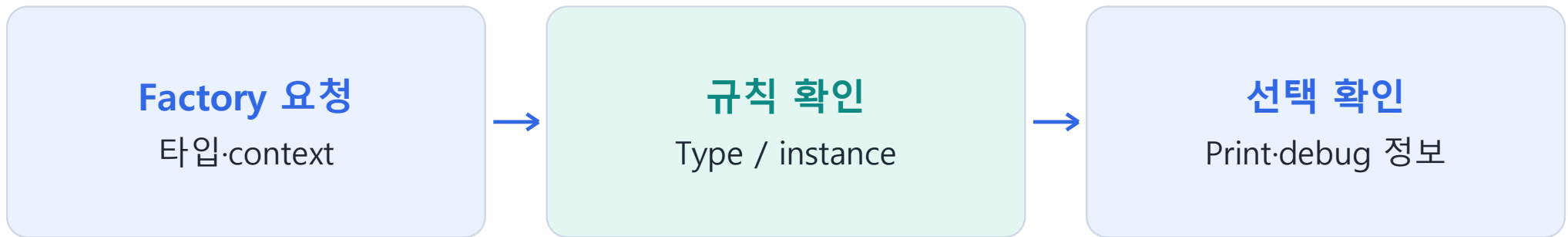
- Object/component utils는 등록 역할이다.
- Field macro는 copy·compare·print·pack 참여와 표시 정책을 정한다.

Copy / clone

- Copy는 기존 목적지에 값을 복사한다.
- Clone은 새 바깥 객체를 만들고 복사한다.
- 내부 객체의 독립성은 복사 정책에 달려 있다.

타입과 경로로 factory 선택을 추적한다

핵심 개념



do_* hook

- do_copy·do_compare·do_print 등은 작업별 사용자 구현이다.
- 상속 필드를 처리하려면 부모 구현을 재사용한다.

타입·경로 디버깅

- Factory 설정은 이미 만든 객체를 교체하지 않는다.
- Print/debug API로 선택 규칙과 경로를 확인하고 매개변수 클래스는 type 기반 등록을 구분한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01  fifo_item a, b, c;  
02  a = fifo_item::type_id::create("a");  
03  fifo_item::type_id::set_type_override(  
04      tagged_item::get_type());  
05  b = fifo_item::type_id::create("b");  
06  c = new("c");
```

코드를 읽는 기준

- tagged_item이 fifo_item을 상속하고 등록돼 있다면 a와 c의 실제 타입은 fifo_item, b는 tagged_item이다.

a는 fifo_item, b는 tagged_item이 된다

코드의 동작



c = new()는 factory 규칙을 거치지 않음

결과 해설

- tagged_item이 fifo_item을 상속하고 등록돼 있다면 a와 c의 실제 타입은 fifo_item, b는 tagged_item이다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Override는 생성 전에 설정한다.

주의 2

- UVM_NOCOMPARE는 비교 제외이며 다른 작업까지 모두 끄는 flag가 아니다.

주의 3

- Clone됐다는 이유만으로 모든 중첩 객체가 독립이라고 가정하지 않는다.

이 장의 핵심

- Factory와 new
- Type / instance override
- Utils와 field
- Copy / clone
- do_* hook
- 타입·경로 디버깅

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Override 설정 뒤 기존 a의 타입은 바뀌는가?

문제 2

- 일치하는 instance override와 type override 중 우선하는 것은?

문제 3

- 복사에 UVM_REFERENCE를 사용하면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- 이후 생성 요청에 적용된다.

해설 2

- Instance override다.

해설 3

- 내부 객체의 handle을 공유할 수 있다.

8장

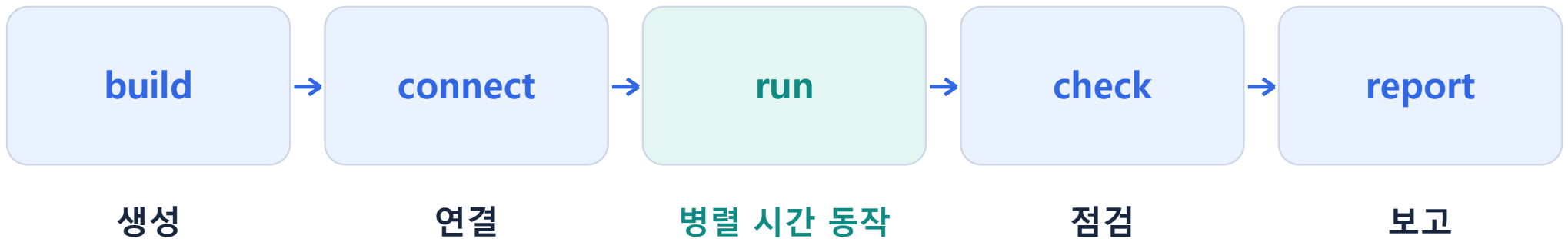
Phase와 Objection

소개

- Phase는 환경 생성·연결·실행·결과 점검을 순서 있게 진행한다.
- Objection은 task phase에 필요한 일이 남아 있음을 알리며, 실제 완료 조건과 연결해야 한다.

구성 요소와 데이터 흐름

구조와 흐름

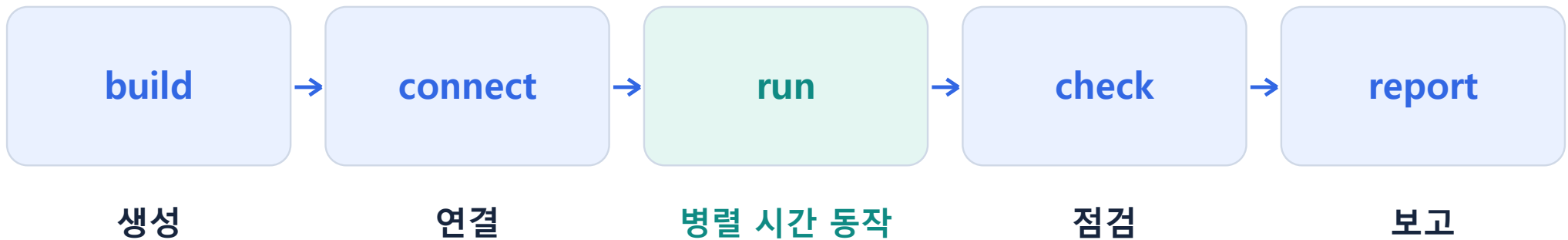


그림을 읽는 기준

- Phase는 환경 생성·연결·실행·결과 점검을 순서 있게 진행한다.
- Objection은 task phase에 필요한 일이 남아 있음을 알리며, 실제 완료 조건과 연결해야 한다.

생성, 연결, 실행, 점검을 순서대로 진행한다

핵심 개념



Build: 위 → 아래 / Connect: 아래 → 위

Phase 책임

- Build는 생성·설정, connect는 연결, run은 시간 동작, check는 결과 점검, report는 요약을 맡는다.

계층 방문 방향

- Build와 final은 top-down이다.
- Connect 등 주요 function phase는 bottom-up이고 run은 component별 병렬 실행한다.

Objection을 실제 작업의 수명과 연결한다

핵심 개념



Task와 병렬성

- Task는 시간을 소비할 수 있다는 뜻이다.
- 병렬 실행은 fork나 UVM scheduler가 만든 별도 프로세스 구조에서 생긴다.

Objection 수명

- Raise는 phase 유지 요청, drop은 그 요청 해제다.
- 모든 관련 objection과 종료 조건이 해소돼야 다음 phase로 진행한다.

미래 요청과 미처리 응답을 함께 확인한다

핵심 개념

stimulus_done = 1

앞으로 보낼 요청 없음

pending_count = 0

미처리 응답·비교 없음

두 조건과 관찰 완료 기준을 함께 확인

완료와 timeout

- Sequence 종료 뒤에도 DUT 응답과 비교가 남을 수 있다.
- 실제 완료 조건을 기다리고 timeout으로 누락을 검출한다.

보조 정책

- Drain time은 추가 여유이며 완료 판정이 아니다.
- Runtime schedule·domain·자동 objection은 실행 context와 종료 정책을 함께 고려한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 phase.raise_objection(this);
02 seq.start(env.agent.sqr);
03 wait (stimulus_done && pending_count == 0);
04 phase.drop_objection(this);
05 // 응답 누락에 대비한 timeout을 병행한다.
```

코드를 읽는 기준

- Stimulus 완료와 미래 입력이 없다는 조건을 함께 정의해야 한다.
- 이 코드의 상태 변수는 환경의 완료 계약을 표현한다.

마지막 비교까지 기다린 뒤 objection을 내린다

코드의 동작



결과 해설

- Stimulus 완료와 미래 입력이 없다는 조건을 함께 정의해야 한다.
- 이 코드의 상태 변수는 환경의 완료 계약을 표현한다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Objection 없이 실행 중인 task만으로 phase가 유지되지는 않는다.

주의 2

- Drop 누락은 정상 종료를 막는다.

주의 3

- Monitor가 phase 종료 후 drop을 기다리면 순환 대기가 생긴다.

이 장의 핵심

- Phase 책임
- 계층 방문 방향
- Task와 병렬성
- Objection 수명
- 완료와 timeout
- 보조 정책

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Task가 실행 중이면 objection 없이도 phase가 유지되는가?

문제 2

- Queue가 순간 비었다면 종료해도 되는가?

문제 3

- Drop은 즉시 $\$finish$ 와 같은가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 그 사실만으로 유지되지 않는다.

해설 2

- 미래 요청·응답이 없는지 함께 확인해야 한다.

해설 3

- 아니다.
- 관련 종료 조건 이후 cleanup phase가 진행된다.

9장

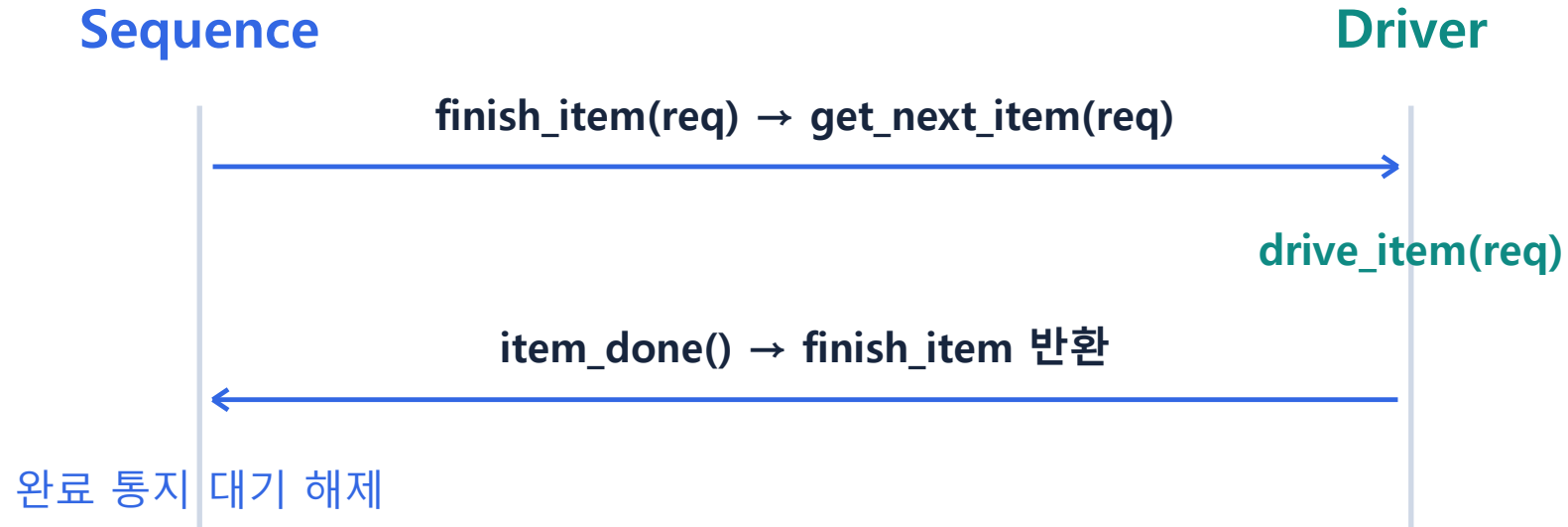
Sequence와 Driver Handshake

소개

- Sequence는 동작 요청을 만들고 driver는 요청을 실제 신호로 수행한다.
- Start_item·finish_item과 get_next_item·item_done 사이의 약속을 이해하면 요청의 생성·전달·완료를 구분할 수 있다.

구성 요소와 데이터 흐름

구조와 흐름

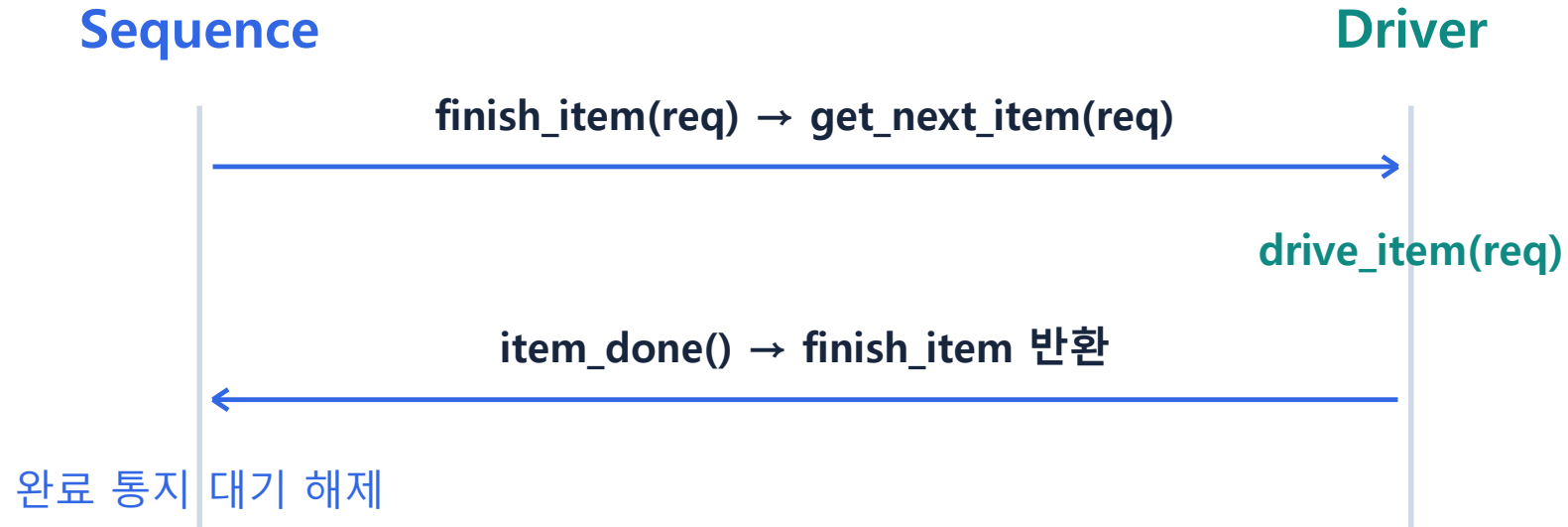


그림을 읽는 기준

- Sequence는 동작 요청을 만들고 driver는 요청을 실제 신호로 수행한다.
- Start_item·finish_item과 get_next_item·item_done 사이의 약속을 이해하면 요청의 생성·전달·완료를 구분할 수 있다.

요청 준비와 완료 통지를 연결한다

핵심 개념



생성과 실행

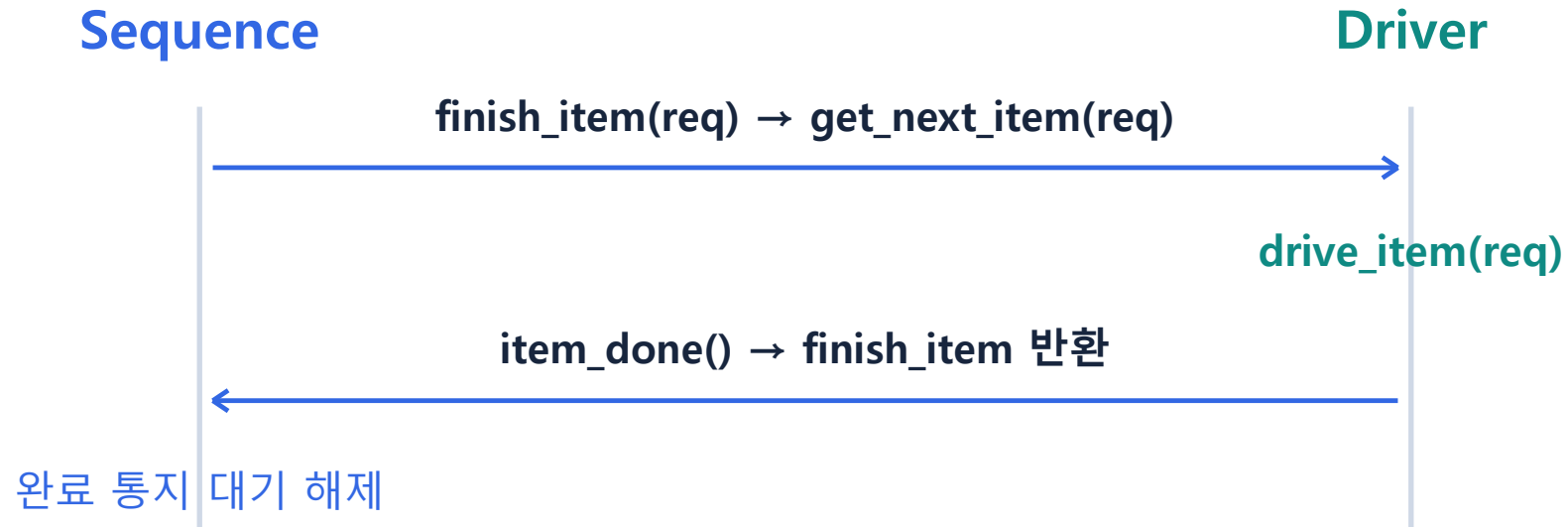
- Sequence create는 객체 생성, start는 body 실행이다.
- Item 생성과 randomization도 서로 다른 작업이다.

Sequence handshake

- Start_item은 허가를 기다린다.
- Finish_item은 준비된 item을 전달하고 완료 통지를 기다린다.

요청 완료 통지와 DUT 응답은 별도다

핵심 개념



Driver handshake

- Get_next_item으로 요청을 받으면 대응하는 item_done이 필요하다.
- 완료 전 같은 경로에서 중복 get_next_item을 호출하지 않는다.

get와 response

- Get 방식은 item을 가져올 때 handshake가 완료된다.
- Optional response 객체와 DUT 결과 검증은 요청 완료 통지와 별개다.

여러 sequence의 요청 차례를 중재한다

핵심 개념

Sequence A

A1 → A2

Sequence B

B1 → B2

중재 예: A1 → B1 → A2 → B2

중재와 순서

- Sequencer는 여러 sequence의 요청 차례를 중재한다.
- 한 sequence 안의 순서를 임의로 뒤집는 기능은 아니다.

Nested / virtual sequence

- 일반 start 호출은 순차 실행, fork/join은 병렬 실행이다.
- Virtual sequence는 여러 sequencer의 시나리오를 조정한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

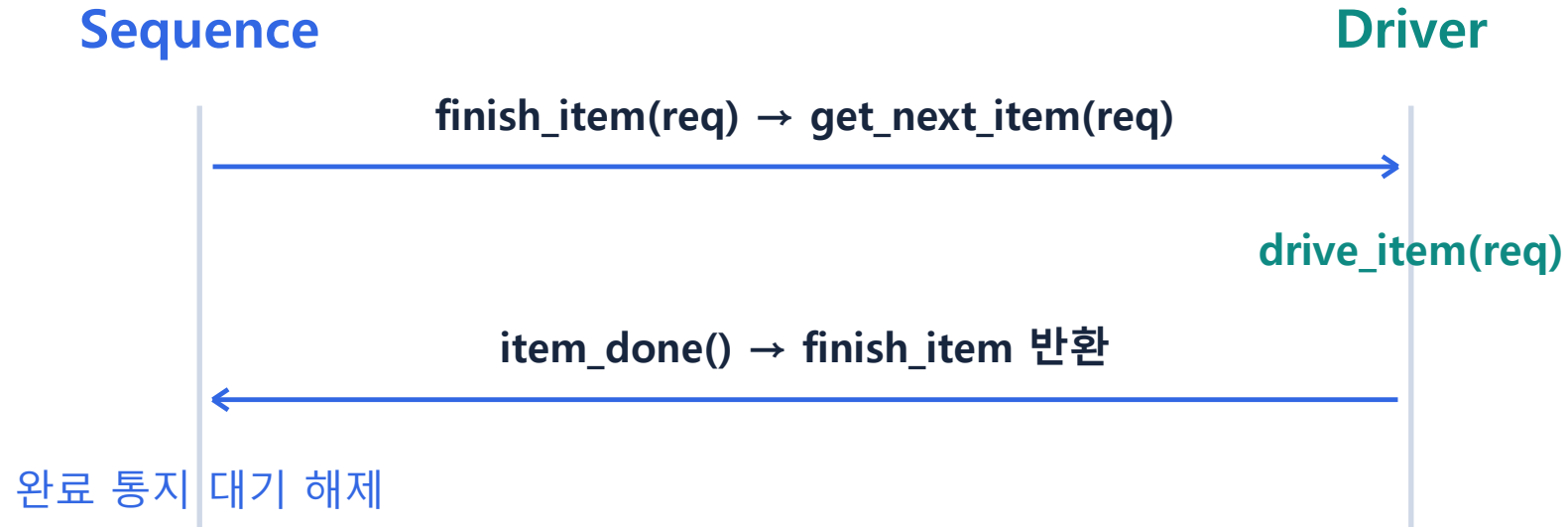
```
01 // Sequence body
02 req = fifo_item::type_id::create("req");
03 start_item(req);
04 if (!req.randomize())
05     `uvm_fatal("RAND", "Randomization failed")
06 finish_item(req);
07 // Driver run_phase 내부
08 seq_item_port.get_next_item(req);
09 drive_item(req);
10 seq_item_port.item_done();
```

코드를 읽는 기준

- 요청 준비는 finish_item 전에 수행한다.
- Driver가 item_done를 빠뜨리면 sequence가 완료를 기다리며 멈출 수 있다.

Item_done이 요청 완료 대기를 해제한다

코드의 동작



결과 해설

- 요청 준비는 finish_item 전에 수행한다.
- Driver가 item_done을 빠뜨리면 sequence가 완료를 기다리며 멈출 수 있다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Finish_item 뒤 필드 변경은 이미 전달한 요청을 뒤늦게 바꾸는 것이다.

주의 2

- Item_done은 모든 DUT 출력의 검증 완료를 뜻하지 않는다.

주의 3

- Response를 기다리는 sequence에는 실제 response 전달 경로가 필요하다.

이 장의 핵심

- 생성과 실행
- Sequence handshake
- Driver handshake
- get와 response
- 중재와 순서
- Nested / virtual sequence

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Sequence 객체를 만들기만 하면 body가 실행되는가?

문제 2

- Item_done이 빠지면 어떤 일이 생기는가?

문제 3

- 한 sequence의 A 다음 B를 sequencer가 B부터 보내는가?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- start가 필요하다.

해설 2

- Finish_item의 완료 대기가 풀리지 않을 수 있다.

해설 3

- 그런 재배열 기능은 아니다.
- 다른 sequence 요청이 끼어들 수는 있다.

10장

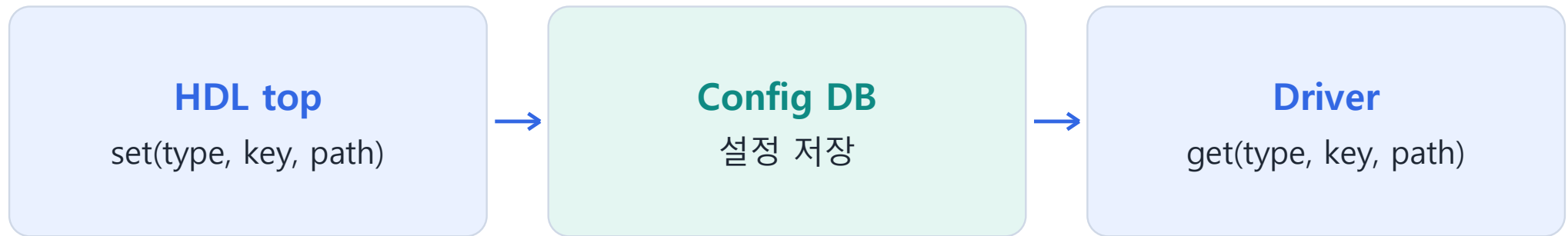
Config DB와 Virtual Interface

소개

- Config DB는 계층 경로에 맞춰 설정을 등록하고 조회하는 저장소다.
- Virtual interface와 configuration object를 전달할 때 타입.이름.경로.시점을 함께 맞춰야 한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- Config DB는 계층 경로에 맞춰 설정을 등록하고 조회하는 저장소다.
- Virtual interface와 configuration object를 전달할 때 타입·이름·경로·시점을 함께 맞춰야 한다.

경로에 맞는 interface 참조를 조회한다

핵심 개념



Virtual interface 전달

- HDL top의 실제 interface instance를 DB에 등록하고 driver가 virtual interface 변수로 조회한다.

Context와 경로

- Context의 전체 경로와 상대 inst_name을 결합한다.
- Get(this, "", ...)는 현재 component 위치에서 조회한다.

타입, key, 경로와 설정 시점을 맞춘다

핵심 개념

일치 기준	set	get
Type	virtual fifo_if	virtual fifo_if
Key	vif	vif
Scope	...agent.drv	현재 drv 위치

타입·이름·시점

- Set/get의 타입과 field_name이 맞아야 한다.
- 필요한 get보다 먼저 설정을 등록하고 get 성공과 null을 따로 검사한다.

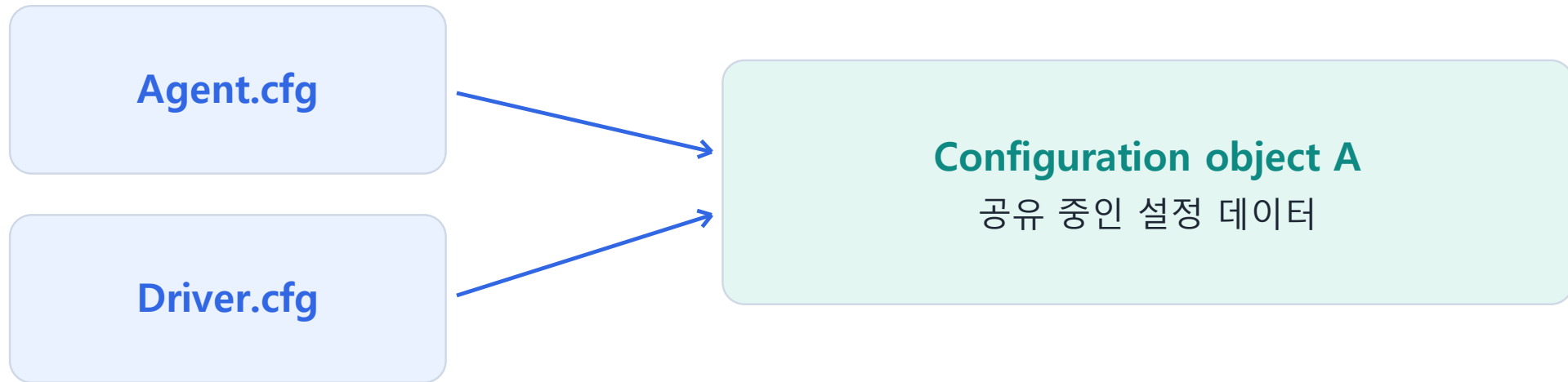
우선순위

- Build 중에는 높은 계층의 설정이 우선한다.
- 같은 precedence에서는 나중 설정이 우선하며 runtime 정책은 build와 구분한다.

새 설정 등록과 기존 참조를 구분한다

핵심 개념

DB에 B를 등록해도 기존 A 참조는 유지



객체 공유

- Configuration object의 handle을 공유하면 필드 수정이 보일 수 있다.
- 새 객체를 DB에 등록해도 기존 handle이 자동 교체되지는 않는다.

범위와 추적

- Wildcard 범위를 필요한 대상에 제한한다.
- Config DB trace와 topology로 실제 경로·타입·등록을 확인한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 // HDL top: run_test() 전에 등록
02 uvm_config_db#(virtual fifo_if)::set(
03     null, "uvm_test_top.env.agent.drv", "vif", fifo_bus);
04 // Driver build_phase
05 if (!uvm_config_db#(virtual fifo_if)::get(
06     this, "", "vif", vif))
07     `uvm_fatal("NO_VIF", "Interface not found")
08 if (vif == null)
09     `uvm_fatal("NULL_VIF", "Interface is null")
```

코드를 읽는 기준

- 타입·key·대상 경로가 일치해야 한다.
- DB에서 찾았다는 것과 유효한 참조를 얻었다는 것은 별도 조건이다.

등록된 interface를 driver의 vif로 받는다

코드의 동작



결과 해설

- 타입·key·대상 경로가 일치해야 한다.
- DB에서 찾았다는 것과 유효한 참조를 얻었다는 것은 별도 조건이다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Handle 변수 이름 대신 create의 instance 이름이 component 경로에 들어간다.

주의 2

- DB의 값 변경이 이미 가져온 scalar 변수를 자동 갱신하지 않는다.

주의 3

- 실제 interface와 virtual 참조를 구분한다.

이 장의 핵심

- Virtual interface 전달
- Context와 경로
- 타입.이름.시점
- 우선순위
- 객체 공유
- 범위와 추적

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Get 성공이면 vif가 반드시 non-null인가?

문제 2

- DB에 새 객체 B를 등록하면 기존 agent의 A 참조는?

문제 3

- Create("drv", this)의 경로 leaf는?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- 별도 null 검사가 필요하다.

해설 2

- 다시 조회·대입하지 않으면 A를 유지한다.

해설 3

- drv다.
- Handle 변수 이름과 별개다.

11장

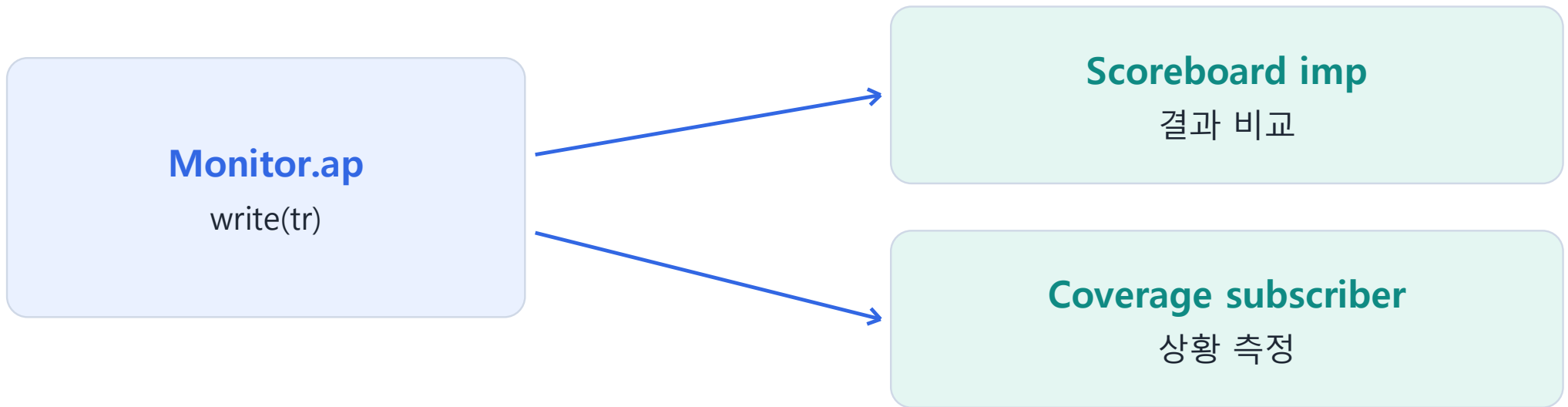
TLM과 Analysis

소개

- TLM은 component 사이에서 약속된 메서드로 transaction을 전달한다.
- 연결과 호출을 구분하고, analysis 방송의 시간·복사 특성을 이해하면 데이터 경로를 정확히 구성할 수 있다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- TLM은 component 사이에서 약속된 메서드로 transaction을 전달한다.
- 연결과 호출을 구분하고, analysis 방송의 시간·복사 특성을 이해하면 데이터 경로를 정확히 구성할 수 있다.

호출점, 중계점, 구현 끝점을 연결한다

핵심 개념



생성·연결·호출

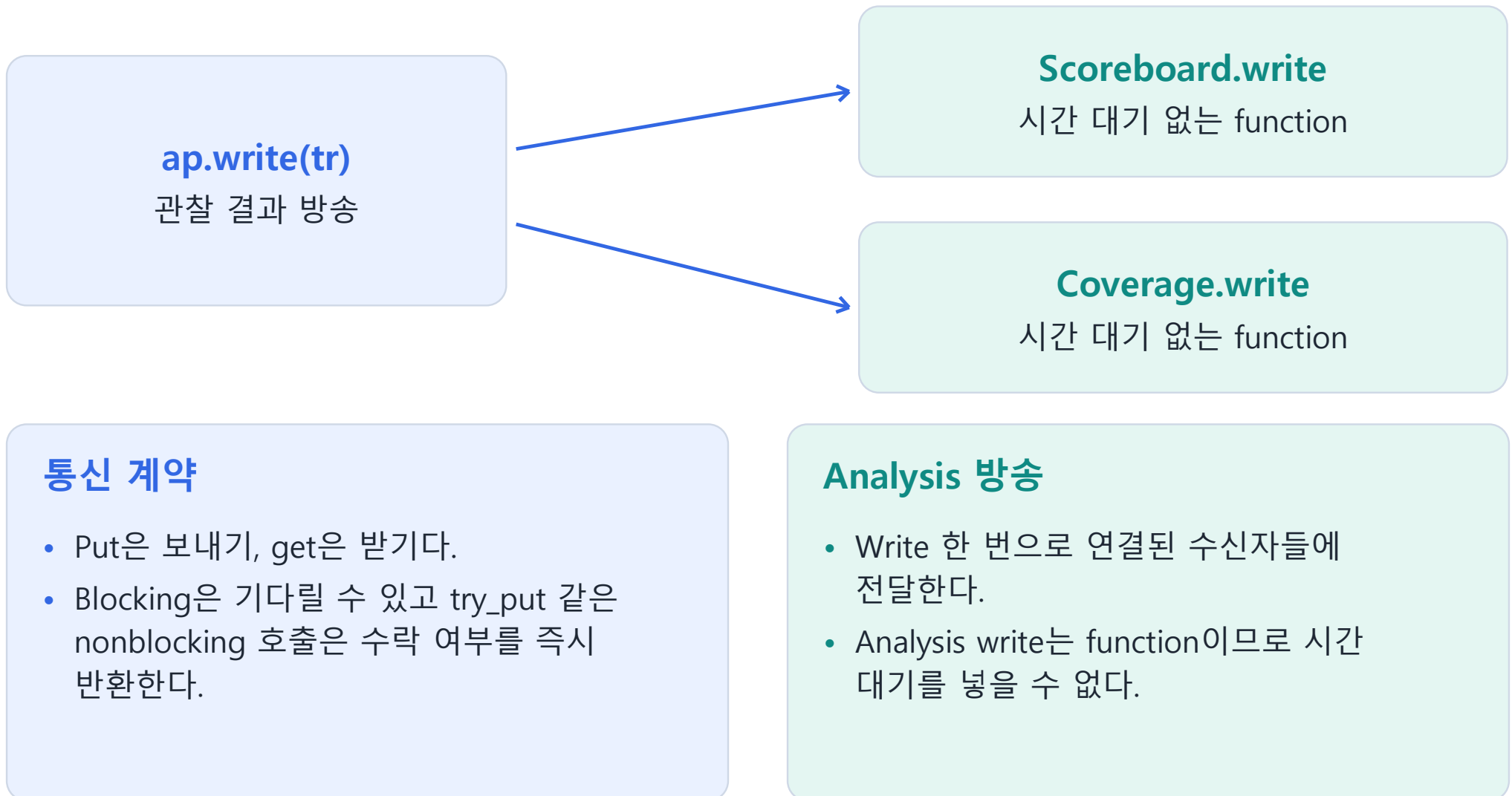
- 단자를 만들고 connect한 뒤 put/get/write 등을 호출해야 실제 통신한다.
- Connect만으로 데이터가 흐르지 않는다.

Port / export / imp

- Port는 호출점, export는 구현 경로 중계, imp는 component의 구현 메서드에 연결하는 끝점이다.

관찰 결과를 여러 수신자에게 전달한다

핵심 개념



전달과 소비 시점을 FIFO로 분리한다

핵심 개념



FIFO도 자동 deep copy를 하지 않는다

Handle과 snapshot

- Class handle은 복사본이 아니다.
- 오래 보관할 데이터는 새 객체에 복사하고 subscriber가 공유 원본을 수정하지 않도록 정책을 정한다.

FIFO와 계층

- Analysis FIFO는 전달과 소비 시점을 분리하며 누적될 수 있다.
- 송신 계층은 child port→parent port, 수신 계층은 export→내부 imp로 중계한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

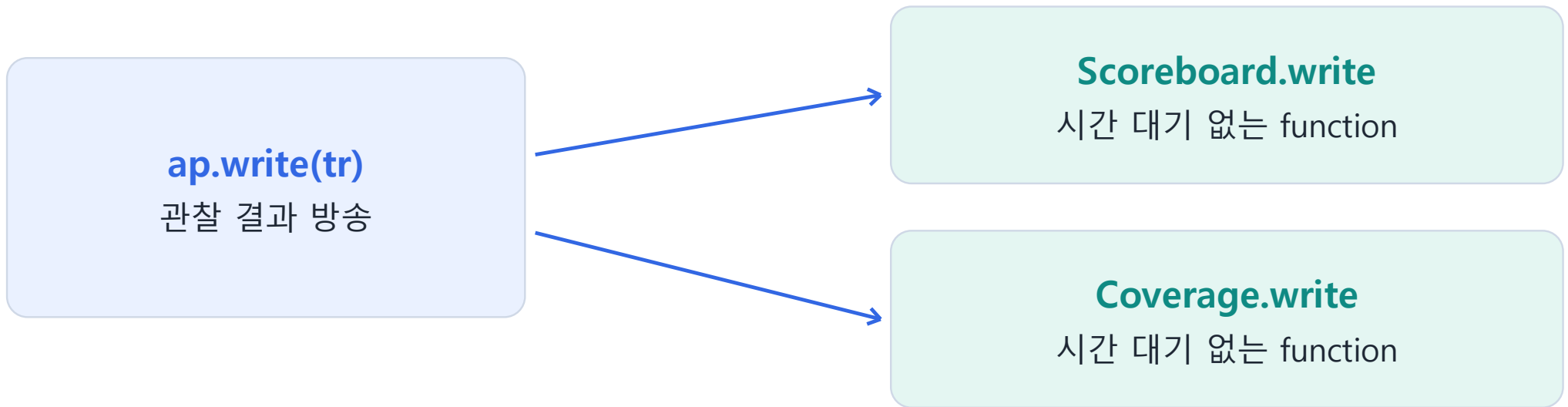
```
01 // env connect_phase
02 mon.ap.connect(sb.analysis_imp);
03 mon.ap.connect(cov.analysis_export);
04 // monitor: 새로운 관찰 결과 발행
05 ap.write(tr);
06 // scoreboard의 수신 메서드
07 function void write(fifo_item tr);
08     process_observation(tr);
09 endfunction
```

코드를 읽는 기준

- Write는 연결된 수신 메서드를 호출한다.
- 느린 시간 동작이 필요하다면 analysis FIFO 등에 넘기고 별도의 task에서 처리한다.

Write 한 번이 여러 수신자에게 전달된다

코드의 동작



결과 해설

- Write는 연결된 수신 메서드를 호출한다.
- 느린 시간 동작이 필요하면 analysis FIFO 등에 넘기고 별도의 task에서 처리한다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Analysis에는 수신자의 ready/수락 handshake가 없다.

주의 2

- Analysis FIFO가 객체를 자동 deep copy하지는 않는다.

주의 3

- 변수 이름이 export여도 선언 타입이 imp면 역할은 imp다.

이 장의 핵심

- 생성·연결·호출
- Port / export / imp
- 통신 계약
- Analysis 방송
- Handle과 snapshot
- FIFO와 계층

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Connect만 했을 때 transaction이 전달되는가?

문제 2

- Write 안에서 @(posedge clk)를 기다릴 수 있는가?

문제 3

- Subscriber 하나가 공유 tr을 수정하면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 호출이 없으므로 전달되지 않는다.

해설 2

- Function이므로 시간 대기는 불가능하다.

해설 3

- 다른 수신자나 보관한 handle에 영향이 생길 수 있다.

12장

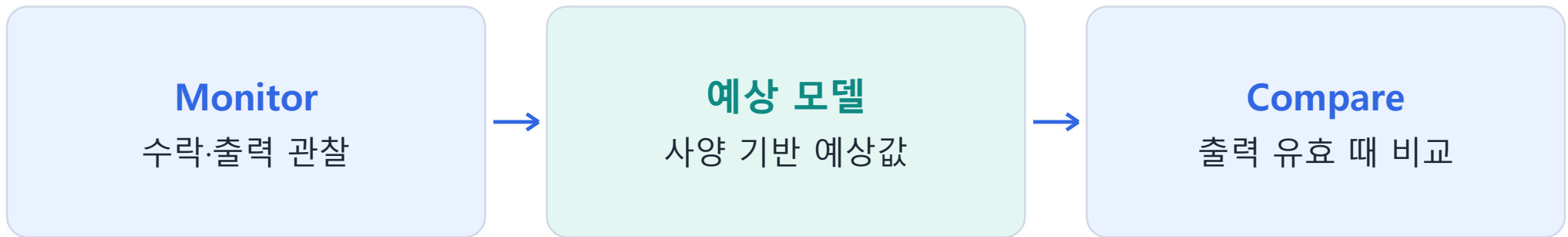
Monitor와 Scoreboard

소개

- Monitor는 실제 신호를 관찰하고 scoreboard는 사양에 맞는 예상 결과를 만든다.
- FIFO 예제로 요청 수락, 출력 latency, reset과 종료 조건을 함께 추적한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- Monitor는 실제 신호를 관찰하고 scoreboard는 사양에 맞는 예상 결과를 만든다.
- FIFO 예제로 요청 수락, 출력 latency, reset과 종료 조건을 함께 추적한다.

수락된 요청만 예상 모델에 반영한다

핵심 개념

쓰기 요청

full = 1 / write = 1

쓰기 수락

거부: expected_q 변화 없음

요청 신호와 수락 여부를 구분

관찰과 검사

- Monitor는 driver의 의도가 아니라 실제 interface를 관찰한다.
- Scoreboard는 예상값과 실제값을 비교한다.

요청과 수락

- Full에서 거부된 쓰기는 예상 queue에 넣지 않는다.
- DUT flag의 정확성은 별도의 예상 상태와 비교한다.

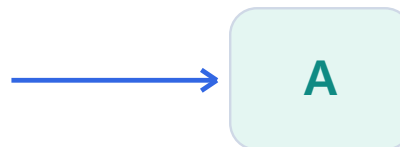
읽기 수락과 결과 비교 사이를 추적한다

핵심 개념

읽기 수락 전 저장 예상값



읽기 수락 뒤 응답 대기



N: A 수락 → N+1: actual과 A 비교

예상값의 출처

- 예상 데이터는 관찰한 입력과 사양에서 만든다.
- DUT 출력값을 예상값으로도 사용하면 자기 비교가 된다.

Latency와 대기 상태

- 읽기 수락 때 예상값을 확보하고 출력 유효 시점에 비교한다.
- 저장 데이터와 응답 대기 상태는 의미가 다르다.

Reset과 동시 요청의 사양을 모델에 반영한다

핵심 개념

Reset에서 취소

이전 pending 응답 정리

동시 read / write

수락 규칙은 DUT 사양 적용

Queue 계산 순서와 DUT 시간 순서를 구분

Reset과 종료

- 취소된 응답과 이전 관찰이 새 데이터와 섞이지 않게 처리한다.
- Sequence 종료 뒤 필요한 응답·최종 상태까지 확인한다.

경계 동시 요청

- Full/empty에서 동시 read/write 수락은 사양에 따른다.
- 모델 코드의 pop→push가 DUT의 시간 순서를 의미하지 않는다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 // 설명 계약: 수락 신호와 출력 유효 정보 제공
02 if (read_accept)
03     pending_reads.push_back(expected_q.pop_front());
04 if (write_accept)
05     expected_q.push_back(observed_wdata);
06 if (read_valid) begin
07     expected = pending_reads.pop_front();
08     if (actual !== expected)
09         `uvm_error("FIFO_DATA", "Read mismatch")
10 end
```

코드를 읽는 기준

- 수락 여부는 pre-state와 사양으로 먼저 결정한다.
- 실제 구현에서는 pop 전 queue 크기, reset 취소와 관찰 순서를 함께 검사한다.

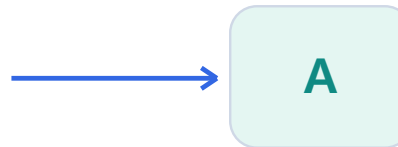
N에서 수락한 A를 N+1에서 비교한다

코드의 동작

읽기 수락 전 저장 예상값



읽기 수락 뒤 응답 대기



N: A 수락 → N+1: actual과 A 비교

결과 해설

- 수락 여부는 pre-state와 사양으로 먼저 결정한다.
- 실제 구현에서는 pop 전 queue 크기, reset 취소와 관찰 순서를 함께 검사한다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- 읽기 수락과 출력 유효는 같은 시점이라고 가정하지 않는다.

주의 2

- 저장 queue가 남아도 의도한 최종 상태라면 정상일 수 있다.

주의 3

- 예상 모델이 DUT의 잘못된 flag를 그대로 따라가지 않게 한다.

이 장의 핵심

- 관찰과 검사
- 요청과 수락
- 예상값의 출처
- Latency와 대기 상태
- Reset과 종료
- 경계 동시 요청

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Full에서 거부된 쓰기를 예상 queue에 넣는가?

문제 2

- 읽기 1사이클 지연이면 N에서 수락한 A는 언제 비교하는가?

문제 3

- 비교할 응답이 남았는데 sequence가 끝났다면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 놓지 않는다.

해설 2

- 계약상 출력이 유효한 $N+1$ 시점이다.

해설 3

- 응답 처리와 완료 조건을 확인한 뒤 종료한다.

13장

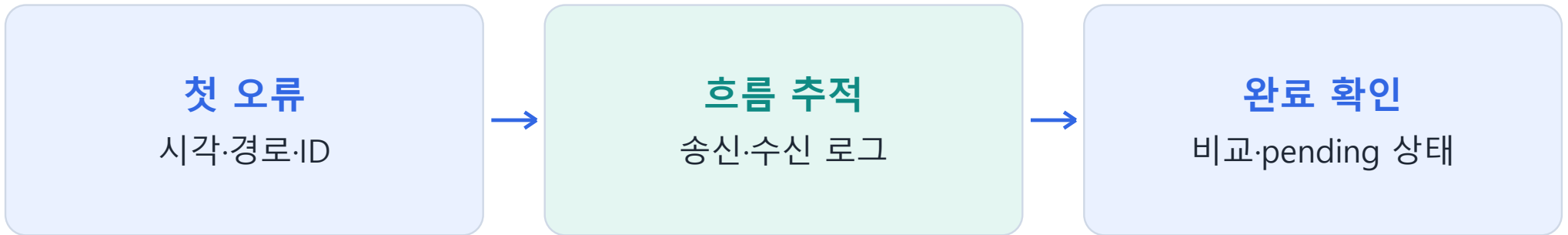
Report와 디버깅

소개

- UVM report는 로그의 종류와 상세도를 관리하고 오류 원인을 추적할 정보를 남긴다.
- 최초 오류부터 실행 경로를 좁히고, 오류 횟수뿐 아니라 실제 비교와 완료 여부를 확인한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- UVM report는 로그의 종류와 상세도를 관리하고 오류 원인을 추적할 정보를 남긴다.
- 최초 오류부터 실행 경로를 좁히고, 오류 횟수뿐 아니라 실제 비교와 완료 여부를 확인한다.

로그 종류와 정보의 상세도를 구분한다

핵심 개념

출력 설정	LOW info	HIGH info
UVM_LOW	표시	숨김
UVM_HIGH	표시	표시

Severity는 종류, verbosity는 정보 상세도

Severity

- Info는 정보, warning은 주의, error는 검증 실패, fatal은 계속 진행할 수 없는 상황에 사용한다.

Verbosity

- 메시지 수준이 출력 설정 이하이면 info가 보인다.
- 설정을 높이면 더 상세한 로그까지 출력한다.

첫 오류에서 이후 오류의 원인을 찾는다

핵심 개념



첫 원인과 뒤의 결과를 함께 추적

ID와 본문

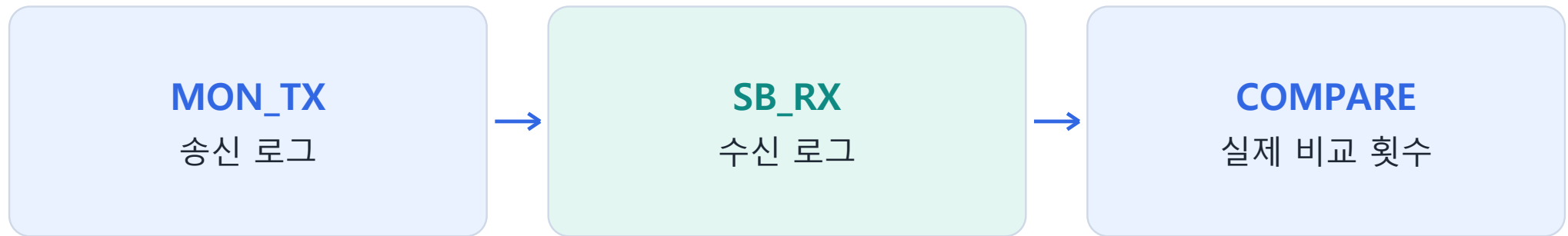
- ID는 종류를 분류한다.
- 본문에는 주소·동작·예상값·실제값·transaction 정보 등 원인 추적에 필요한 값을 남긴다.

최초 오류

- NO_VIF 같은 설정 오류가 뒤의 데이터 누락을 유발할 수 있다.
- 최초 오류와 이후 결과의 관계부터 확인한다.

수신 로그 뒤 실제 비교까지 확인한다

핵심 개념



데이터 경로

- Monitor 송신과 scoreboard 수신 로그를 비교한다.
- 수신 후에는 유효 조건·예상값·비교 횟수를 확인한다.

검증 완료

- 오류 0개가 비교 수행을 보장하지 않는다.
- 필요한 비교 횟수, pending 응답, 최종 상태와 timeout을 함께 확인한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01  `uvm_info("TRACE", "Item received", UVM_HIGH)
02  if (actual !== expected)
03      `uvm_error("REG_DATA",
04          $sformatf("addr=%0h expected=%0h actual=%0h",
05                      addr, expected, actual))
06  // 실제 계층과 타입 확인
07  uvm_top.print_topology();
```

코드를 읽는 기준

- ID는 같은 종류의 메시지에 재사용하고 매번 달라지는 값은 본문에 넣는다.
- Topology 출력은 구조 구성 이후 phase에 배치한다.

예상값과 실제값의 불일치를 로그로 남긴다

코드의 동작

expected = A5

예상 모델 값

actual = 0F

관찰한 실제 값

REG_DATA 오류: 주소·expected·actual 기록

결과 해설

- ID는 같은 종류의 메시지에 재사용하고 매번 달라지는 값은 본문에 넣는다.
- Topology 출력은 구조 구성 이후 phase에 배치한다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Error macro는 일반 info verbosity로 숨겨지는 로그가 아니다.

주의 2

- Factory override는 타입을 바꾸며 create 이름을 자동 변경하지 않는다.

주의 3

- 비교 0건·error 0건을 검증 성공으로 판단하지 않는다.

이 장의 핵심

- Severity
- Verbosity
- ID와 본문
- 최초 오류
- 데이터 경로
- 검증 완료

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- LOW 설정에서 HIGH info는 출력되는가?

문제 2

- 필요 비교 10건 중 8건만 완료하고 error 0이면?

문제 3

- 수신 로그는 있는데 비교 횟수가 0이라면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 일반 설정에서는 숨겨진다.

해설 2

- 수행한 비교는 통과했지만 전체는 미완료다.

해설 3

- 내부 유효 조건과 예상값·비교 실행 경로를 확인한다.

14장

Coverage와 Assertion

소개

- Coverage는 계획한 상황을 시험했는지 측정하고 assertion은 조건에 따른 규칙을 검사한다.
- Scoreboard·functional coverage·code coverage와 함께 사용해야 검증 결과의 의미를 판단할 수 있다.

구성 요소와 데이터 흐름

구조와 흐름

Scoreboard

데이터 결과

Functional coverage

상황 발생

Assertion

시간·신호 규칙

Code coverage

RTL 실행 정도

그림을 읽는 기준

- Coverage는 계획한 상황을 시험했는지 측정하고 assertion은 조건에 따른 규칙을 검사한다.
- Scoreboard·functional coverage·code coverage와 함께 사용해야 검증 결과의 의미를 판단할 수 있다.

결과 검사와 상황 측정을 함께 사용한다

핵심 개념

Scoreboard

결과가 맞는가?

Assertion

규칙을 지켰는가?

Functional coverage

상황이 발생했는가?

Code coverage

RTL이 실행됐는가?

네 가지 역할

- Scoreboard는 데이터 결과, assertion은 규칙, functional coverage는 기능 상황, code coverage는 구현 실행 정도를 확인한다.

Covergroup와 bins

- Covergroup에 coverpoint와 cross를 묶는다.
- Bin은 목표 값·범위·변화를 구분하는 항목이며 sampling 때 기록한다.

같은 시점의 조건 조합을 기록한다

핵심 개념

empty	read 요청	같은 sampling
0	0	데이터 있음 / 요청 없음
0	1	정상 읽기 요청
1	0	비어 있음 / 요청 없음
1	1	Empty 읽기 요청

같은 시점의 cross

- Empty와 read를 따로 관찰해도 empty 읽기 조합을 시험한 것은 아니다.
- 상태와 요청을 같은 판단 시점에 sampling한다.

시간 규칙

- Concurrent assertion은 clock의 sampled 값으로 property를 평가한다.
- $|=>$ 는 다음 clock 검사, disable iff는 reset 중 평가 중단에 사용한다.

상황 발생과 규칙 검사 기회를 함께 본다

핵심 개념

Empty 읽기 발생

규칙 검사 기회 있음

Empty 읽기 미발생

Implication 뒤 검사 없음

상황 발생 coverage와 assertion 결과를 함께 본다

Vacuous success

- 시작 조건이 없으면 implication의 뒤 규칙을 시험하지 않고 통과할 수 있다.
- 조건 발생 coverage와 함께 해석한다.

Coverage hole

- 가능·미시험이면 자극을 보강하고, 이미 발생했다면 sampling을 확인한다.
- 불가능한 조합은 사양 근거로 목표에서 제외한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01  covergroup fifo_cg with function sample(bit e, bit r);  
02      cp_empty : coverpoint e;  
03      cp_read  : coverpoint r;  
04      empty_read : cross cp_empty, cp_read;  
05  endgroup  
06  // 클래스 constructor에서 cg = new();  
07  // 같은 판단 시점의 값을 전달  
08  cg.sample(empty_before, read_request);
```

코드를 읽는 기준

- 두 bit coverpoint의 기본 bins와 cross로 네 조합을 측정한다.
- 거부된 empty 읽기 요청도 관찰해야 해당 조합을 기록할 수 있다.

Empty와 read를 같은 sampling에 기록한다

코드의 동작

empty	read 요청	같은 sampling
0	0	데이터 있음 / 요청 없음
0	1	정상 읽기 요청
1	0	비어 있음 / 요청 없음
1	1	Empty 읽기 요청

결과 해설

- 두 bit coverpoint의 기본 bins와 cross로 네 조합을 측정한다.
- 거부된 empty 읽기 요청도 관찰해야 해당 조합을 기록할 수 있다.

같은 sampling 시점의 조합

개념 비교

empty	read 요청	상황
0	0	데이터 있음 / 요청 없음
0	1	데이터 있을 때 읽기
1	0	비어 있음 / 요청 없음
1	1	Empty 읽기 요청

해석

- 별도 cycle에서 두 조건을 관찰한 것과 같은 시점의 조합 발생을 구분한다.

Empty 읽기 뒤 포인터가 유지되는가?

Assertion의 시간 규칙



핵심 코드

```
assert property (@(posedge clk) disable iff (!rst_n)
    (empty && rd_en) | => $stable(rd_ptr));
```

사양 가정: empty 읽기 거부.
Reset 외의 다른 포인터 변경 원인은 없다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- Coverage 100%는 오류가 없다는 보장이 아니다.

주의 2

- 수락된 읽기만 수집하면 거부되는 empty 요청을 놓칠 수 있다.

주의 3

- Assertion의 timing과 reset은 실제 DUT 계약에 맞춰 정의한다.

이 장의 핵심

- 네 가지 역할
- Covergroup와 bins
- 같은 시점의 cross
- 시간 규칙
- Vacuous success
- Coverage hole

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Coverage가 채워지면 데이터 결과도 맞는가?

문제 2

- Assertion failure 0인데 시작 조건이 한 번도 없었다면?

문제 3

- Functional coverage 100%의 범위는?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- Scoreboard 등으로 별도 검사한다.

해설 2

- 해당 상황의 규칙을 확인했다고 볼 수 없다.

해설 3

- 정의한 coverage 목표의 범위다.
- 계획 누락은 별도 검토한다.

15장

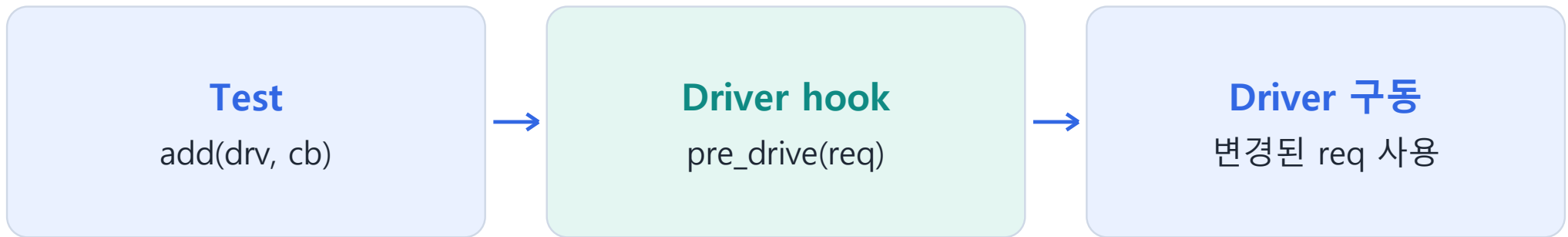
Callback

소개

- Callback은 준비된 호출 지점에서 별도 객체의 동작을 실행하는 확장 방법이다.
- Driver의 기본 구동 흐름을 유지하면서 테스트별 오류 주입·추가 기록 등을 조합할 수 있다.

구성 요소와 데이터 흐름

구조와 흐름

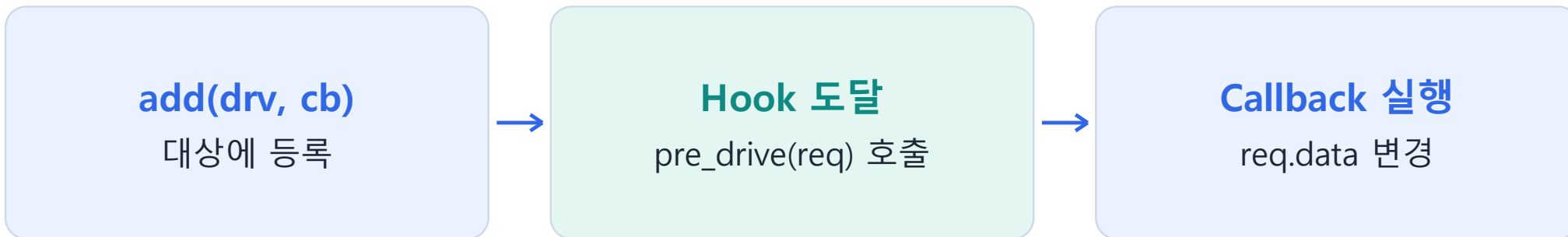


그림을 읽는 기준

- Callback은 준비된 호출 지점에서 별도 객체의 동작을 실행하는 확장 방법이다.
- Driver의 기본 구동 흐름을 유지하면서 테스트별 오류 주입·추가 기록 등을 조합할 수 있다.

등록된 callback은 hook에서 실행된다

핵심 개념



등록과 실행

- Add는 어느 대상에 callback을 붙일지 정한다.
- 실제 실행에는 component가 hook을 호출해야 한다.

Callback 객체

- Uvm_callback을 상속한 기본 틀에 virtual 메서드를 정의하고 자식 callback에서 필요한 동작을 구현한다.

대상과 callback 타입을 짝짓는다

핵심 개념

대상 타입

fifo_driver

Callback 타입

fifo_driver_cb

uvm_register_cb로 짝 등록 / uvm_do_callbacks로 호출

타입 짝과 macro

- Uvm_register_cb는 대상 타입과 callback 타입을 짝짓는다.
- Uvm_do_callbacks는 현재 대상의 활성 callback 메서드를 호출한다.

Handle 수정

- Pre_drive에 전달한 transaction handle은 원본을 참조한다.
- Callback의 필드 변경을 이후 driver 구동이 사용한다.

Callback 범위와 실행 순서가 결과를 바꾼다

핵심 개념

Factory override

새 객체의 실제 타입 선택

Callback

준비된 hook의 실행 동작

A: F0 대입 → B: XOR FF → 최종 0F

적용 대상과 순서

- 특정 driver instance에 등록할 수 있다.
- 여러 callback이 같은 데이터를 바꾸면 등록·실행 순서가 결과에 영향을 준다.

Factory와 선택

- Factory는 생성 타입을 선택한다.
- Callback은 준비된 hook의 추가 동작을 구성한다.
- 전체 프로토콜 변경은 driver 확장도 검토한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

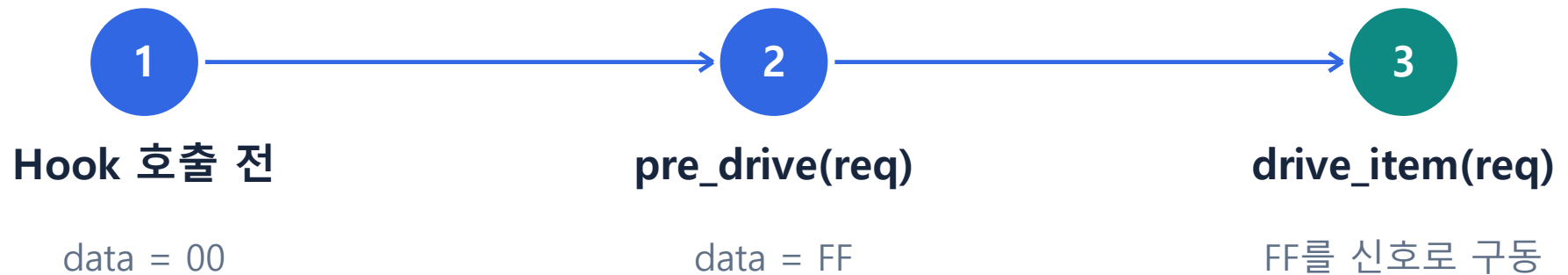
```
01 // Test: driver 생성 후 등록
02 cb = corrupt_data_cb::type_id::create("cb");
03 uvm_callbacks#(fifo_driver, fifo_driver_cb)::add(
04     env.agent.drv, cb);
05 // Driver: 요청을 받은 뒤 구동 직전
06 `uvm_do_callbacks(
07     fifo_driver, fifo_driver_cb, pre_drive(req))
08 drive_item(req);
```

코드를 읽는 기준

- Callback의 pre_drive가 req.data를 FF로 바꾸면 driver는 변경된 값을 사용한다.
- Add만 있고 hook 호출이 없으면 실행되지 않는다.

구동 직전 hook에서 data를 FF로 바꾼다

코드의 동작



결과 해설

- Callback의 pre_drive가 req.data를 FF로 바꾸면 driver는 변경된 값을 사용한다.
- Add만 있고 hook 호출이 없으면 실행되지 않는다.

Factory override와 Callback

개념 비교

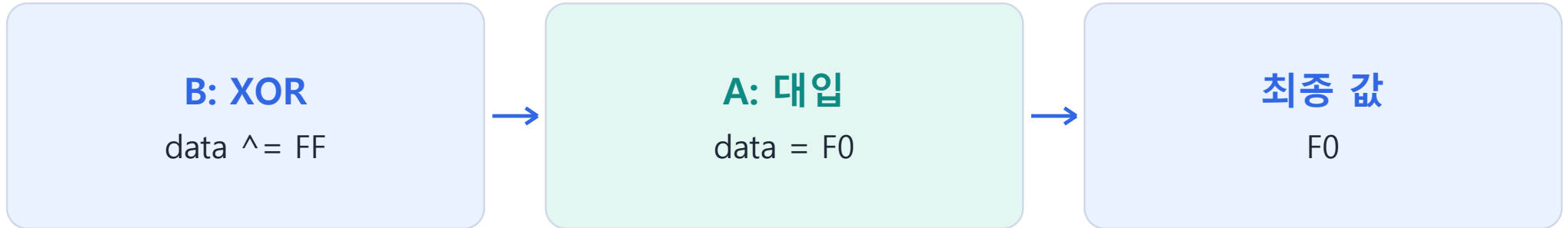
관점	Factory override	Callback
바꾸는 대상	생성할 실제 타입	Hook의 실행 동작
적용 시점	객체 생성 때	Hook 호출 때
기존 객체	소급 교체 없음	대상에 등록 가능

해석

- 전체 구동 방식과 특정 지점의 추가 동작을 구분해 선택한다.

같은 데이터에 두 동작을 적용한다

Callback의 실행 순서



해석

- 같은 transaction을 수정하므로 실행 순서가 최종 값을 바꾼다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- uvm_register_cb만으로 callback 객체가 생성·등록되지는 않는다.

주의 2

- 한 instance에 등록한 callback이 다른 instance에 자동 적용되지는 않는다.

주의 3

- 오류 주입 뒤 scoreboard는 실제 수락된 입력과 사양을 기준으로 예상값을 만든다.

이 장의 핵심

- 등록과 실행
- Callback 객체
- 타입 짝과 macro
- Handle 수정
- 적용 대상과 순서
- Factory와 선택

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Add만 하고 hook 호출이 없으면 데이터가 바뀌는가?

문제 2

- A가 F0 대입, B가 FF XOR, $A \rightarrow B$ 면 결과는?

문제 3

- 변경된 FF 쓰기가 full에서 거부되면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 아니다.
- Callback 메서드가 실행되지 않는다.

해설 2

- 0F다.
- 반대 순서면 마지막 대입으로 F0이다.

해설 3

- Scoreboard 예상 queue에 추가하지 않는다.

16장

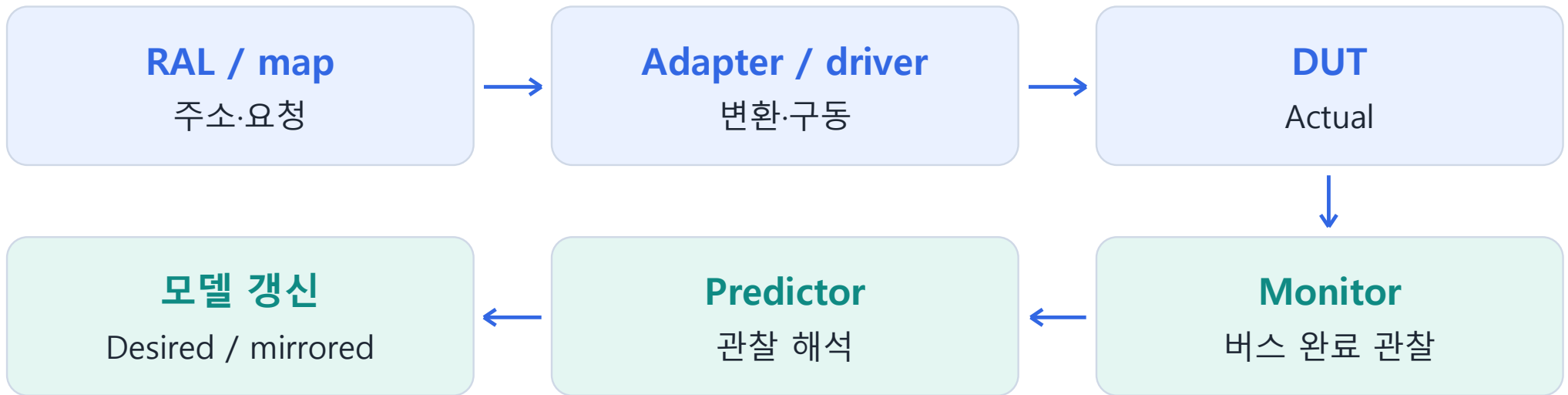
Register Abstraction Layer

소개

- RAL은 DUT 레지스터를 이름과 필드 단위로 표현하는 모델이다.
- 주소 배치와 접근 변환, 실제 버스 동작, 관찰 결과의 prediction을 연결하고 모델 값과 DUT 값을 구분해야 한다.

구성 요소와 데이터 흐름

구조와 흐름



그림을 읽는 기준

- RAL은 DUT 레지스터를 이름과 필드 단위로 표현하는 모델이다.
- 주소 배치와 접근 변환, 실제 버스 동작, 관찰 결과의 prediction을 연결하고 모델 값과 DUT 값을 구분해야 한다.

모델 값과 DUT 값을 따로 추적한다

핵심 개념

호출 / 사건	Desired	Mirrored	Actual
초기 상태	0	0	0
set(1)	1	0	0
update + 완료·prediction	1	1	1

모델과 DUT

- Desired는 원하는 모델 값, mirrored는 모델이 알고 있는 값, actual은 DUT의 실제 값이다.
- 자동으로 항상 같지는 않다.

접근 메서드

- Set/get은 desired 설정·조회다.
- Get_mirrored_value는 캐시 조회, mirror는 DUT 읽기·갱신이며 UVM_CHECK로 비교를 요청한다.

필드 비트 범위와 버스 주소를 배치한다

핵심 개념

mode: 폭 3 / LSB 4 → bits [6:4]



Map base 1000 + offset 04 = bus address 1004

Update와 write

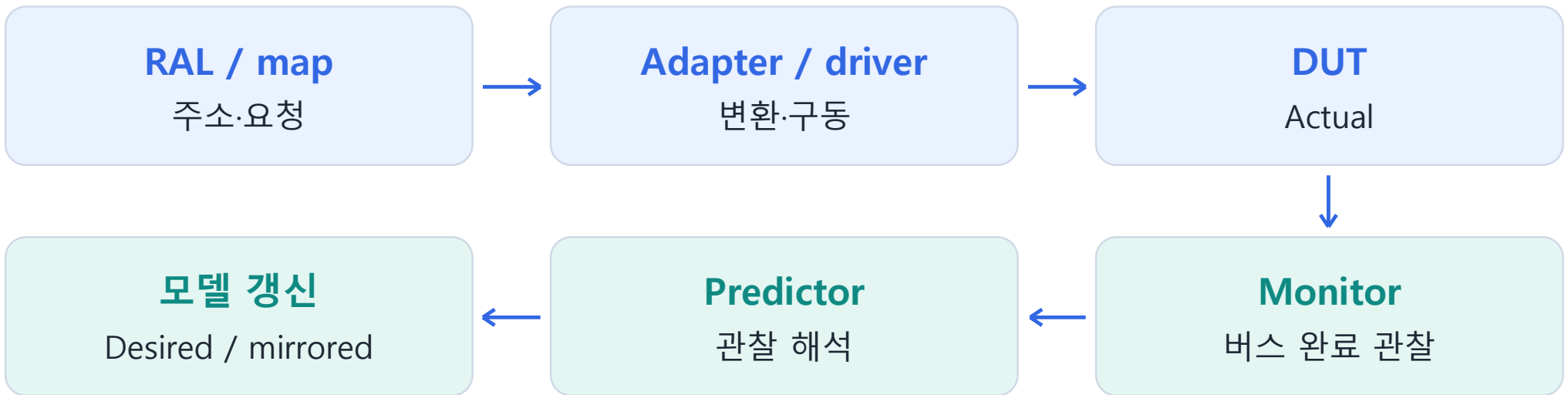
- 단순 RW에서 update는 desired/mirrored 차이로 필요 쓰기를 판단한다.
- Write는 지정 값을 쓴다.
- Update는 DUT를 먼저 읽지 않는다.

모델 구조

- Block 안에 register와 field를 구성하고 map에 주소를 등록한다.
- Field는 폭과 LSB로 범위를 정의하며 주소 등록은 add_reg가 맡는다.

실제 관찰을 prediction 경로로 반영한다

핵심 개념



Adapter와 predictor

- Adapter는 reg2bus/bus2reg 변환이다.
- Predictor는 monitor의 관찰을 모델에 반영한다.
- Auto prediction과 중복 적용하지 않도록 구성한다.

Reset과 접근 속성

- 모델 reset은 desired/mirrored, 실제 reset은 DUT를 초기화한다.
- RW·RO·W1C 정책과 frontdoor/backdoor의 경로 차이를 반영한다.

코드에서 중요한 호출을 따라간다

핵심 예제

핵심 코드

```
01 // 실제 접근 경로
02 model.default_map.set_sequencer(agent.sqr, adapter);
03 // 관찰 결과의 prediction 경로
04 predictor.map = model.default_map;
05 predictor.adapter = adapter;
06 agent.mon.ap.connect(predictor.bus_in);
07 model.default_map.set_auto_predict(0);
08 // 성공한 접근과 prediction 완료를 전제
09 model.CTRL.set(1);
10 model.CTRL.update(status);
```

코드를 읽는 기준

- Reset 후 모두 0인 단순 RW 모델에서 성공한 update와 prediction이 끝나면 desired·mirrored·actual은 모두 1이다.

Set과 update의 값 변화를 추적한다

코드의 동작

호출 / 사건	Desired	Mirrored	Actual
초기 상태	0	0	0
set(1)	1	0	0
update + 완료·prediction	1	1	1

결과 해설

- Reset 후 모두 0인 단순 RW 모델에서 성공한 update와 prediction이 끝나면 desired·mirrored·actual은 모두 1이다.

세 가지 값과 set / update

개념 비교

호출 / 상태	Desired	Mirrored	Actual
Reset 후	0	0	0
set(1)	1	0	0
update 성공 + prediction	1	1	1

해석

- 단순 RW 기준이다.
- Update는 DUT를 읽지 않고 모델의 차이로 쓰기를 판단한다.

Mirror는 실제 읽기, 조회는 캐시 읽기

개념 비교

메서드	DUT 접근	역할
get()	없음	Desired 조회
get_mirrored_value()	없음	기억한 Mirrored 조회
mirror(status)	읽기	모델 갱신
mirror(status, UVM_CHECK)	읽기	기존 Mirror와 비교 후 갱신

해석

- 읽기 성공 후 단순 RW의 desired와 mirrored가 읽은 값으로 반영된다.

모델 reset과 실제 reset

개념 비교

모두 1에서 수행	Desired	Mirrored	Actual
모델 reset만	0	0	1
DUT reset만	1	1	0
둘 다 reset	0	0	0

해석

- 정의된 reset 값 0, 다른 관찰·prediction 없음, 실제 reset 완료를 가정한다.

주소 배치와 필드 범위

개념 비교

정의	값	결과
Map base / offset	1000 + 04	버스 주소 1004
mode 폭 / LSB	3 / 4	bits [6:4]
count 폭 / LSB	8 / 8	bits [15:8]
Map 등록	add_reg(STATUS, 04, RO)	명시적으로 주소 배치

해석

- 바이트 주소 map을 가정한다.
- Register 생성만으로 주소가 자동 등록되지는 않는다.

두 prediction 방식

개념 비교

방식	갱신 근거	RAL 밖의 버스 접근
Auto prediction	해당 map의 접근 결과	그 경로만으로는 알 수 없음
Monitor 기반 predictor	실제 관찰한 접근	관찰·해석되면 반영 가능

해석

- 버스에 나타나지 않는 DUT 내부 자체 변화는 별도 읽기·관찰 정책이 필요하다.

RW / RO / W1C

개념 비교

속성	쓰기 의미	예시
RW	쓰는 값을 저장	1111에 0101 → 0101
RO	버스 쓰기로 변경 불가	값은 DUT 동작으로 변할 수 있음
W1C	1을 쓴 비트 지움	1111에 0101 → 1010

해석

- 다른 내부 변화가 없고 쓰기가 정상 적용된 필드 예제다.

Frontdoor와 backdoor

개념 비교

경로	접근 방식	확인할 수 있는 것
Frontdoor	Map·adapter·버스 agent	실제 버스 접근 동작
Backdoor	HDL 경로 등 내부 접근	내부 저장값 확인·설정

해석

- Backdoor로 내부 값을 바꿔도 버스 주소 해석 오류를 검증한 것은 아니다.

혼동하기 쉬운 지점을 짚는다

주의점과 핵심 정리

주의 1

- DUT 쓰기 성공만으로 mirrored가 갱신된다고 가정하지 않는다.

주의 2

- 모델 reset은 DUT reset 신호를 구동하지 않는다.

주의 3

- Backdoor 성공만으로 버스 주소 해석·handshake가 정상이라고 판단하지 않는다.

이 장의 핵심

- 모델과 DUT
- 접근 메서드
- Update와 write
- 모델 구조
- Adapter와 predictor
- Reset과 접근 속성

그림과 코드의 동작을 설명해본다

확인 문제와 해설

문제 1

- Set(1)만 호출하면 세 값은?
- 초기값은 모두 0.

문제 2

- 모두 1일 때 reset 값 0의 model.reset만 하면?

문제 3

- W1C 현재값 1111에 0101을 쓰면?

개념과 결과를 연결한다

확인 문제와 해설

해설 1

- 단순 RW에서 desired=1, mirrored=0, actual=0이다.

해설 2

- 0, 0, 1이다.
- DUT actual은 그대로다.

해설 3

- 1을 쓴 비트를 지워 1010이 된다.

참고 자료와 예제 기준

참고 자료

UVM 1.2 Class Reference

- Factory, phase, TLM, Callback, register API의 기준 자료

UVM Class Reference 열기

예제의 해석

- API 예제는 UVM 1.2 기준이다.
- FIFO의 수락, 출력 latency와 reset은 명시된 사양 가정을 따른다.
- RAL의 접근 결과는 field 속성과 prediction 정책을 함께 해석한다.
- 실제 환경에는 사용하는 library와 DUT 사양을 적용한다.